

Recapture Analysis: Final

John D. Robinson

2026-08-22

Contents

Recapture Analysis: Combined	1
Load COLONY and PLINK2 Results	2
Evaluate Technical Replicates - Error Rate, Kinship	2
Check for Odd Patterns	12
Compare Inferred Recaptures	18
Final Recapture Calls	22
Check for Odd Patterns in the FINAL clone assignments	35
Assign Final Sex	43
Compare kinship coefficients	45
Final Summary	48
Save the output!	49

Recapture Analysis: Combined

In this markdown, we'll combine the results from COLONY and PLINK2 to identify recaptures across the samples processed by RAPiD Genomics / LGC and GTSeek. We'll also estimate naive genotyping error rates, assess kinship coefficients between technical replicates, evaluate concordance in sex inferences from genotypic data, assign clone IDs for all samples, and assign final sexes for all unique genotypes.

The outputs from this step will be used subsequently to create a capture history object for SCR modeling and to subsample to one sample per unique genotype for population genetic analyses.

The strategy used to combine inferences from COLONY and PLINK2 is as follows:

1. Start with COLONY clones as recaptures - most (if not all) of these are also recovered by PLINK
2. For each unique COLONY clone, ask if PLINK provides compelling evidence to move the clone to a larger group
 - Mean kinship across pairwise comparisons of clone A and clone B needs to be > 0.45 (based on the range of kinship coefficients seen in technical replicates)
 - At least one pairwise comparison between the clones needs to be based on a substantial number of shared markers (> 50)

Load COLONY and PLINK2 Results

First, load the files with results from COLONY and PLINK2.

```
#Read the metadata
smd <- read.csv("../sampleinfo/sampleMetadata_sexGT.csv",
                header = TRUE)

#Read the COLONY output
col.out <- read.table("CGA.BestClone", header = TRUE)

#Read the PLINK output
kingrel <- read.table("recapPLINK.kin0", header = FALSE)
colnames(kingrel) <- c("id1", "id2", "nsnp", "hehet", "ibs0", "kinship")

nrow(kingrel) #Number of rows (pairwise comparisons) in PLINK output...
```

```
## [1] 1408681
```

```
choose(sum(!smd$FILT),2) # should match choose(# retained samples, 2)
```

```
## [1] 1408681
```

Evaluate Technical Replicates - Error Rate, Kinship

First pull the technical replicate samples out of the larger data file. Start by extracting genotypes (in 012 format from the combined file that we used for the recapture analysis.

```
ulimit -n 3000
vcftools --vcf ../snppfiltering/finalSCR.recode.vcf --012 --out finalSCR
```

VCftools - 0.1.17

(C) Adam Auton and Anthony Marcketta 2009

Parameters as interpreted:

```
--vcf ../snppfiltering/finalSCR.recode.vcf
--012
--out finalSCR
```

```
Warning: Expected at least 2 parts in INFO entry: ID=AF,Number=A,Type=Float,Description="Estimated allele frequency in the population"
Warning: Expected at least 2 parts in INFO entry: ID=PRO,Number=1,Type=Float,Description="Reference allele"
Warning: Expected at least 2 parts in INFO entry: ID=PAO,Number=A,Type=Float,Description="Alternate allele"
Warning: Expected at least 2 parts in INFO entry: ID=SRP,Number=1,Type=Float,Description="Strand balance"
Warning: Expected at least 2 parts in INFO entry: ID=SAP,Number=A,Type=Float,Description="Strand balance"
Warning: Expected at least 2 parts in INFO entry: ID=AB,Number=A,Type=Float,Description="Allele balance"
Warning: Expected at least 2 parts in INFO entry: ID=ABP,Number=A,Type=Float,Description="Allele balance"
Warning: Expected at least 2 parts in INFO entry: ID=RPP,Number=A,Type=Float,Description="Read Placement"
Warning: Expected at least 2 parts in INFO entry: ID=RPPR,Number=1,Type=Float,Description="Read Placement"
Warning: Expected at least 2 parts in INFO entry: ID=EPP,Number=A,Type=Float,Description="End Placement"
Warning: Expected at least 2 parts in INFO entry: ID=EPPR,Number=1,Type=Float,Description="End Placement"
```

Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=CIGAR,Number=A,Type=String,Description="The extend
After filtering, kept 1679 out of 1679 Individuals
Writing 012 matrix files ... Done.
After filtering, kept 1226 out of a possible 1226 Sites
Run Time = 1.00 seconds

Now read this into R and subset to the technical replicate samples

```
reps <- smd[which(smd$is.rep == TRUE),]
length(unique(sub("R$", "", reps$ugaID)))
```

```
## [1] 60
```

```
nrow(reps) #60 replicated samples = 120 rows
```

```
## [1] 120
```

```
genos <- read.table("finalSCR.012", header = FALSE)
genos.ind <- read.table("finalSCR.012.indv", header = FALSE)
genos[,1] <- genos.ind[,1]

repl.out <- data.frame(ugaID = unique(sub("R$", "", reps$ugaID)),
                      vcfID1=NA, vcfID2=NA,
                      nSNPgt1=NA, nSNPgt2=NA,
                      filt1=NA, filt2=NA,
                      colclone1=NA, colclone2=NA,
                      is.clonePLINK=NA, is.cloneCOLONY=NA,
                      nSNPshared=NA, nSNPmatch=NA,
                      kinship=NA, GTservice=NA)
for(i in 1:nrow(repl.out)) {
  repl.out$GTservice[i] <- reps$GTservice[which(reps$ugaID == repl.out$ugaID[i])][1]

  #Replicates use the same UGA ID in LGC data...
  if(repl.out$GTservice[i] == "LGC") {
    #Record vcfIDs
    repl.out$vcfID1[i] <- reps$vcfID[which(reps$ugaID==repl.out$ugaID[i])][1]
    repl.out$vcfID2[i] <- reps$vcfID[which(reps$ugaID==repl.out$ugaID[i])][2]
    #But append an "R" in GTSeek data
  } else if(repl.out$GTservice[i] == "GTSeek") {
    repl.out$vcfID1[i] <- reps$vcfID[which(reps$ugaID==repl.out$ugaID[i])]
    repl.out$vcfID2[i] <- reps$vcfID[which(reps$ugaID==paste0(repl.out$ugaID[i], "R"))]
  }

  #Record number of SNPs genotyped
  repl.out$nSNPgt1[i] <- smd$nSNPgt[which(smd$vcfID == repl.out$vcfID1[i])]
  repl.out$nSNPgt2[i] <- smd$nSNPgt[which(smd$vcfID == repl.out$vcfID2[i])]
```

```

#Record filtered status - if TRUE, no need to look for matches
repl.out$filt1[i] <- smd$FILT[which(smd$vcfID == repl.out$vcfID1[i])]
repl.out$filt2[i] <- smd$FILT[which(smd$vcfID == repl.out$vcfID2[i])]

if(repl.out$GTservice[i] == "LGC") {
  #Record clone IDs
  if(repl.out$filt1[i] == FALSE) {
    repl.out$colclone1[i] <- col.out$CloneIndex[grep(repl.out$vcfID1[i],
                                                    col.out[,3], fixed=TRUE)]
  }
  if(repl.out$filt2[i] == FALSE) {
    repl.out$colclone2[i] <- col.out$CloneIndex[grep(repl.out$vcfID2[i],
                                                    col.out[,3], fixed = TRUE)]
  }
} else if(repl.out$GTservice[i] == "GTseek") { #Need to account for the overlap in vcfIDs for reps
  if(repl.out$filt2[i] == FALSE) {
    repl.out$colclone2[i] <- col.out$CloneIndex[grep(repl.out$vcfID2[i],
                                                    col.out[,3], fixed = TRUE)]
  }
  if(repl.out$filt1[i] == FALSE) {
    tmp <- col.out$CloneIndex[grep(repl.out$vcfID1[i],
                                    col.out[,3], fixed = TRUE)]

    if(length(tmp) > 1) {
      repl.out$colclone1[i] <- tmp[which(!tmp %in% repl.out$colclone2[i])]
      rm(tmp)
    } else {
      repl.out$colclone1[i] <- tmp
      rm(tmp)
    }
  }
}

if(repl.out$filt1[i] == repl.out$filt2[i] & repl.out$filt1[i] == FALSE) {
  #Record kinship estimate
  repl.out$kinship[i] <- kingrel$kinship[kingrel$id1 %in% c(repl.out$vcfID1[i],
                                                            repl.out$vcfID2[i]) &
                                         kingrel$id2 %in% c(repl.out$vcfID1[i],
                                                            repl.out$vcfID2[i])]

  repl.out$nSNPshared[i] <- kingrel$n SNP[kingrel$id1 %in% c(repl.out$vcfID1[i],
                                                            repl.out$vcfID2[i]) &
                                         kingrel$id2 %in% c(repl.out$vcfID1[i],
                                                            repl.out$vcfID2[i])]

  #Check if identified as recapture via PLINK
  if(!is.na(repl.out$kinship[i])) {
    if(repl.out$kinship[i] > 0.354) {
      repl.out$is.clonePLINK[i] <- TRUE
    } else {
      repl.out$is.clonePLINK[i] <- FALSE
    }
  }
} else {
  repl.out$is.clonePLINK[i] <- NA
}

```

```

}

#Check if identified as recapture via COLONY
if(repl.out$colclone1[i] == repl.out$colclone2[i]) {
  repl.out$is.cloneCOLONY[i] <- TRUE
} else {
  repl.out$is.cloneCOLONY[i] <- FALSE
}

tmp.geno <- genos[genos$V1 %in% c(repl.out$vcfID1[i], repl.out$vcfID2[i]),-1]
ovrlp <- which(colSums(tmp.geno == -1) == 0)
if(length(ovrlp) > 0) {
  #If as.matrix(), even single SNP overlaps are OK!
  tmp.geno <- as.matrix(tmp.geno[,ovrlp])
  repl.out$nSNPmatch[i] <- sum(tmp.geno[1,] == tmp.geno[2,])
} else {
  repl.out$nSNPmatch[i] <- NA
}

rm(tmp.geno, ovrlp)
}
}

```

```

#Some high-level summaries for the pairs of replicated samples
sum(is.na(repl.out$is.clonePLINK))

```

```
## [1] 17
```

```
sum(is.na(repl.out$is.cloneCOLONY))
```

```
## [1] 17
```

```
sum(is.na(repl.out$is.clonePLINK) & is.na(repl.out$is.cloneCOLONY))
```

```
## [1] 17
```

```
repl.out[is.na(repl.out$is.clonePLINK),]
```

##	ugaID	vcfID1	vcfID2	nSNPgt1	nSNPgt2	filt1
## 2	121-23	UGA_173101_P015_WB11	UGA_173101_P016_WA12	1	82	TRUE
## 7	306-23	UGA_173101_P015_WH09	UGA_173101_P015_WH12	0	0	TRUE
## 14	14-C	UGA_173101_P018_WA02	UGA_173101_P018_WG06	1174	1	FALSE
## 17	280-23	UGA_173101_P018_WD05	UGA_173101_P019_WC10	22	20	TRUE
## 21	134-24	UGA_173101_P019_WE07	UGA_173101_P019_WE09	0	1040	TRUE
## 26	434-24	UGA_173101_P019_WG10	UGA_173101_P019_WH01	1	51	TRUE
## 37	517-24	GTS_517-24	GTS_517-24R	NA	NA	TRUE
## 38	716-23	GTS_716-23	GTS_716-23R	NA	NA	TRUE
## 39	726-24	GTS_726-24	GTS_726-24R	NA	NA	TRUE
## 46	1384-24	GTS_1384-24	GTS_1384-24R	NA	NA	TRUE
## 52	2926-24	GTS_2926-24	GTS_2926-24R	NA	NA	TRUE

```
## 54 396-24          GTS_396-24          GTS_396-24R      113      NA FALSE
## 55 424-23          GTS_424-23          GTS_424-23R      NA       NA  TRUE
## 56 191-24          GTS_191-24          GTS_191-24R      NA       NA  TRUE
## 57 466-23          GTS_466-23          GTS_466-23R      NA       NA  TRUE
## 58 286-23          GTS_286-23          GTS_286-23R       3       NA  TRUE
## 60 311-23          GTS_311-23          GTS_311-23R     69       NA  TRUE
##      filt2 colclone1 colclone2 is.clonePLINK is.cloneCOLONY nSNPshared nSNPmatch
## 2      TRUE      NA      NA      NA      NA      NA      NA      NA
## 7      TRUE      NA      NA      NA      NA      NA      NA      NA
## 14     TRUE     16      NA      NA      NA      NA      NA      NA
## 17     TRUE      NA      NA      NA      NA      NA      NA      NA
## 21    FALSE      NA     32      NA      NA      NA      NA      NA
## 26     TRUE      NA      NA      NA      NA      NA      NA      NA
## 37     TRUE      NA      NA      NA      NA      NA      NA      NA
## 38     TRUE      NA      NA      NA      NA      NA      NA      NA
## 39     TRUE      NA      NA      NA      NA      NA      NA      NA
## 46     TRUE      NA      NA      NA      NA      NA      NA      NA
## 52     TRUE      NA      NA      NA      NA      NA      NA      NA
## 54     TRUE     272      NA      NA      NA      NA      NA      NA
## 55     TRUE      NA      NA      NA      NA      NA      NA      NA
## 56     TRUE      NA      NA      NA      NA      NA      NA      NA
## 57     TRUE      NA      NA      NA      NA      NA      NA      NA
## 58     TRUE      NA      NA      NA      NA      NA      NA      NA
## 60     TRUE      NA      NA      NA      NA      NA      NA      NA
##      kinship GTservice
## 2      NA      LGC
## 7      NA      LGC
## 14     NA      LGC
## 17     NA      LGC
## 21     NA      LGC
## 26     NA      LGC
## 37     NA      GTSeek
## 38     NA      GTSeek
## 39     NA      GTSeek
## 46     NA      GTSeek
## 52     NA      GTSeek
## 54     NA      GTSeek
## 55     NA      GTSeek
## 56     NA      GTSeek
## 57     NA      GTSeek
## 58     NA      GTSeek
## 60     NA      GTSeek
```

#NAs come from cases where one or both members of a pair is filtered from the dataset

#PLINK recognizes samples as clones?

```
table(repl.out$is.clonePLINK)
```

```
##
## FALSE  TRUE
##      1    42
```

```
repl.out[which(repl.out$is.clonePLINK == FALSE),]
```

```
##      ugaID          vcfID1          vcfID2 nSNPgt1 nSNPgt2 filt1 filt2
## 1 35-SGA UGA_173101_P015_WB01 UGA_173101_P015_WB03      614      870 FALSE FALSE
##   colclone1 colclone2 is.clonePLINK is.cloneCOLONY nSNPshared nSNPmatch
## 1         74      118          FALSE          FALSE      519      393
##   kinship GTservice
## 1 0.309172      LGC
```

```
#All but 35-SGA
```

```
#COLONY recognizes samples as clones?
```

```
table(repl.out$is.cloneCOLONY)
```

```
##
## FALSE  TRUE
##      4    39
```

```
repl.out[which(repl.out$is.cloneCOLONY == FALSE),]
```

```
##      ugaID          vcfID1          vcfID2 nSNPgt1 nSNPgt2 filt1 filt2
## 1 35-SGA UGA_173101_P015_WB01 UGA_173101_P015_WB03      614      870 FALSE FALSE
## 29 227-23 UGA_173101_P021_WH06 UGA_173101_P023_WC12     1156     1134 FALSE FALSE
## 41 935-23      GTS_935-23      GTS_935-23R      123      123 FALSE FALSE
## 53 151-24      GTS_151-24      GTS_151-24R      122      118 FALSE FALSE
##   colclone1 colclone2 is.clonePLINK is.cloneCOLONY nSNPshared nSNPmatch
## 1         74      118          FALSE          FALSE      519      393
## 29         14      190           TRUE          FALSE     1092      940
## 41        270      303           TRUE          FALSE      122      122
## 53        270      33           TRUE          FALSE      117      117
##   kinship GTservice
## 1 0.309172      LGC
## 29 0.424453      LGC
## 41 0.500000    GTSeek
## 53 0.500000    GTSeek
```

```
#35-SGA, 227-23, 935-23, and 151-24
```

```
#How often do the programs agree??
```

```
table(repl.out$is.clonePLINK[which(repl.out$is.cloneCOLONY == TRUE)])
```

```
##
## TRUE
##    39
```

```
table(repl.out$is.cloneCOLONY[which(repl.out$is.clonePLINK == TRUE)])
```

```
##
## FALSE  TRUE
##      3    39
```

```
repl.out[which(repl.out$is.cloneCOLONY == FALSE &
               repl.out$is.clonePLINK == TRUE),]
```

```
##      ugaID          vcfID1          vcfID2 nSNPgt1 nSNPgt2 filt1 filt2
## 29 227-23 UGA_173101_P021_WH06 UGA_173101_P023_WC12    1156    1134 FALSE FALSE
## 41 935-23          GTS_935-23          GTS_935-23R     123     123 FALSE FALSE
## 53 151-24          GTS_151-24          GTS_151-24R     122     118 FALSE FALSE
##      colclone1 colclone2 is.clonePLINK is.cloneCOLONY nSNPshared nSNPmatch
## 29          14        190          TRUE          FALSE        1092         940
## 41          270        303          TRUE          FALSE         122         122
## 53          270         33          TRUE          FALSE         117         117
##      kinship GTservice
## 29 0.424453      LGC
## 41 0.500000      GTSeek
## 53 0.500000      GTSeek
```

So all COLONY matches in this set are confirmed by PLINK, but PLINK finds a handful of matches that COLONY does not. Obviously, all (or almost all) of the matches in this set are real. These come from duplicated samples. It's somewhat worrisome that COLONY does not match pairs of samples with matching genotypes at 117 and 122 mutually genotyped loci (see the comparisons above). It's less surprising that COLONY doesn't match the 227-23 sample, given the number of genotype mismatches there. Note that PLINK *does* recover the former two sample pairs (151-24 and 935-23) as matches, so the combined approach we're using would classify them as members of the same clone!

With the mismatches we see for 227-23 and 35-SGA, it seems best to exclude those samples from the dataset.

Mismatching Genotypes?

Now let's look at the number of mismatches between replicated genotype calls. This will give us a crude estimate of the genotyping error rate.

```
repl.out$mismatch <- repl.out$nSNPshared-repl.out$nSNPmatch
repl.out$errorRate <- repl.out$mismatch/repl.out$nSNPshared

range(repl.out$mismatch, na.rm = TRUE)
```

```
## [1] 0 152
```

```
range(repl.out$errorRate, na.rm = TRUE)
```

```
## [1] 0.0000000 0.2427746
```

```
#Naive estimate of genotyping error rate is the average proportion of mismatches?
mean(repl.out$errorRate, na.rm = TRUE)
```

```
## [1] 0.01934416
```

```
# 1.93% including all samples
mean(repl.out$errorRate[which(repl.out$GTservice == "LGC")], na.rm=TRUE)
```

```
## [1] 0.03294675
```

```
# 3.29% for LGC samples
mean(repl.out$errorRate[which(repl.out$GTservice == "GTSeek")], na.rm = TRUE)
```

```
## [1] 0.0004516712
```

```
# < 0.1% for GTSeek
```

```
#Excluding samples that are suspicious (where COLONY doesn't find a match)
mean(repl.out$errorRate[which(repl.out$is.cloneCOLONY==TRUE)])
```

```
## [1] 0.01153411
```

```
# 1.15% with only "good" pairs
```

```
mean(repl.out$errorRate[which(repl.out$is.cloneCOLONY==TRUE &
                               repl.out$GTservice == "LGC")])
```

```
## [1] 0.01920435
```

```
# 1.92% for LGC matching samples
```

```
mean(repl.out$errorRate[which(repl.out$is.cloneCOLONY==TRUE &
                               repl.out$GTservice == "GTSeek")])
```

```
## [1] 0.0005081301
```

```
# again <0.1% for GTSeek
```

Error rates are 1-3% for the capture-seq (LGC) dataset, but substantially lower for GT-seq (GTSeek) data. The interesting portions of the table comparing technical replicates...

```
repl.out[,c("ugaID", "nSNPg1", "nSNPg2", "filt1", "filt2", "nSNPshared",
            "nSNPmatch", "kinship", "mismatch", "errorRate")]
```

##	ugaID	nSNPg1	nSNPg2	filt1	filt2	nSNPshared	nSNPmatch	kinship	mismatch
## 1	35-SGA	614	870	FALSE	FALSE	519	393	0.309172	126
## 2	121-23	1	82	TRUE	TRUE	NA	NA	NA	NA
## 3	8-23	1217	1185	FALSE	FALSE	1183	1176	0.496555	7
## 4	47-SGA	1077	1199	FALSE	FALSE	1068	1056	0.491848	12
## 5	23-C	1167	1208	FALSE	FALSE	1156	1147	0.495500	9
## 6	58-SGA	1111	1086	FALSE	FALSE	1025	1005	0.485030	20
## 7	306-23	0	0	TRUE	TRUE	NA	NA	NA	NA
## 8	166-24	1200	1211	FALSE	FALSE	1191	1185	0.497253	6
## 9	68-NGA	1217	1207	FALSE	FALSE	1205	1200	0.496867	5
## 10	21-23	868	919	FALSE	FALSE	728	689	0.463346	39
## 11	251-23	1220	1222	FALSE	FALSE	1220	1216	0.498102	4
## 12	331-23	619	1188	FALSE	FALSE	617	586	0.469246	31
## 13	409-23	859	929	FALSE	FALSE	724	685	0.470808	39
## 14	14-C	1174	1	FALSE	TRUE	NA	NA	NA	NA
## 15	241-23	1219	1220	FALSE	FALSE	1218	1214	0.498054	4
## 16	358-24	1222	1222	FALSE	FALSE	1222	1222	0.500000	0
## 17	280-23	22	20	TRUE	TRUE	NA	NA	NA	NA
## 18	50-23	1185	1222	FALSE	FALSE	1185	1175	0.495220	10
## 19	358-23	1222	1222	FALSE	FALSE	1220	1217	0.497887	3
## 20	5-24	1000	876	FALSE	FALSE	781	741	0.470930	40
## 21	134-24	0	1040	TRUE	FALSE	NA	NA	NA	NA
## 22	219-24	1221	1143	FALSE	FALSE	1143	1128	0.493719	15
## 23	277-24	1214	1209	FALSE	FALSE	1203	1194	0.495223	9
## 24	397-24	1188	1194	FALSE	FALSE	1171	1157	0.491822	14
## 25	423-24	1085	1144	FALSE	FALSE	1042	1022	0.487661	20
## 26	434-24	1	51	TRUE	TRUE	NA	NA	NA	NA
## 27	1-C	1159	1189	FALSE	FALSE	1137	1123	0.493191	14
## 28	17-C	1087	1166	FALSE	FALSE	1050	1020	0.481436	30
## 29	227-23	1156	1134	FALSE	FALSE	1092	940	0.424453	152
## 30	389-24	1203	642	FALSE	FALSE	641	600	0.453196	41
## 31	306-24	1176	1216	FALSE	FALSE	1176	1170	0.497238	6
## 32	930-23	122	123	FALSE	FALSE	122	122	0.500000	0
## 33	564-24	122	122	FALSE	FALSE	122	122	0.500000	0
## 34	781-23	123	123	FALSE	FALSE	122	122	0.500000	0
## 35	798-23	122	123	FALSE	FALSE	122	122	0.500000	0

```

## 36 651-23      122      123 FALSE FALSE      122      122 0.500000      0
## 37 517-24      NA      NA  TRUE  TRUE      NA      NA      NA      NA
## 38 716-23      NA      NA  TRUE  TRUE      NA      NA      NA      NA
## 39 726-24      NA      NA  TRUE  TRUE      NA      NA      NA      NA
## 40 980-24      125      124 FALSE FALSE      124      124 0.500000      0
## 41 935-23      123      123 FALSE FALSE      122      122 0.500000      0
## 42 1306-24     123      123 FALSE FALSE      122      122 0.500000      0
## 43 567-24      122      123 FALSE FALSE      122      122 0.500000      0
## 44 881-24      123      118 FALSE FALSE      118      118 0.500000      0
## 45 2972-24     122      123 FALSE FALSE      120      120 0.500000      0
## 46 1384-24     NA      NA  TRUE  TRUE      NA      NA      NA      NA
## 47 933-24      120      121 FALSE FALSE      118      118 0.500000      0
## 48 1522-24     124      123 FALSE FALSE      123      122 0.492188      1
## 49 3058-24     123      123 FALSE FALSE      122      122 0.500000      0
## 50 2665-24     122      123 FALSE FALSE      121      121 0.500000      0
## 51 2869-24     125      124 FALSE FALSE      124      124 0.500000      0
## 52 2926-24     NA      NA  TRUE  TRUE      NA      NA      NA      NA
## 53 151-24      122      118 FALSE FALSE      117      117 0.500000      0
## 54 396-24      113      NA FALSE TRUE      NA      NA      NA      NA
## 55 424-23      NA      NA  TRUE  TRUE      NA      NA      NA      NA
## 56 191-24      NA      NA  TRUE  TRUE      NA      NA      NA      NA
## 57 466-23      NA      NA  TRUE  TRUE      NA      NA      NA      NA
## 58 286-23      3       NA  TRUE  TRUE      NA      NA      NA      NA
## 59 500-24      122      123 FALSE FALSE      122      122 0.500000      0
## 60 311-23     69      NA  TRUE  TRUE      NA      NA      NA      NA
##      errorRate
## 1 0.242774566
## 2      NA
## 3 0.005917160
## 4 0.011235955
## 5 0.007785467
## 6 0.019512195
## 7      NA
## 8 0.005037783
## 9 0.004149378
## 10 0.053571429
## 11 0.003278689
## 12 0.050243112
## 13 0.053867403
## 14      NA
## 15 0.003284072
## 16 0.000000000
## 17      NA
## 18 0.008438819
## 19 0.002459016
## 20 0.051216389
## 21      NA
## 22 0.013123360
## 23 0.007481297
## 24 0.011955594
## 25 0.019193858
## 26      NA
## 27 0.012313105
## 28 0.028571429
## 29 0.139194139
## 30 0.063962559
## 31 0.005102041
## 32 0.000000000
## 33 0.000000000
## 34 0.000000000
## 35 0.000000000
## 36 0.000000000
## 37      NA
## 38      NA
## 39      NA
## 40 0.000000000
## 41 0.000000000
## 42 0.000000000
## 43 0.000000000
## 44 0.000000000
## 45 0.000000000
## 46      NA
## 47 0.000000000
## 48 0.008130081
## 49 0.000000000
## 50 0.000000000
## 51 0.000000000
## 52      NA
## 53 0.000000000
## 54      NA
## 55      NA
## 56      NA
## 57      NA
## 58      NA
## 59 0.000000000
## 60      NA

```

Kinship?

The range of kinship coefficients for (good) technical replicates will be used below as a threshold for combining COLONY and PLINK inferences. Let's calculate that now.

```

#Range of kinship values between replicates
range(repl.out$kinship, na.rm = TRUE)

```

```
## [1] 0.309172 0.500000
```

```
#Average kinship of "good" pairs
mean(repl.out$kinship[which(repl.out$is.cloneCOLONY==TRUE)])
```

```
## [1] 0.4926236
```

```
range(repl.out$kinship[which(repl.out$is.cloneCOLONY==TRUE)])
```

```
## [1] 0.453196 0.500000
```

```
#So we can set the filter pretty high... >0.45 for recaps
```

```
#Replicated samples that would not meet this threshold...
repl.out[which(repl.out$kinship < 0.45),]
```

```
##      ugaID          vcfID1          vcfID2 nSNPgt1 nSNPgt2 filt1 filt2
## 1  35-SGA UGA_173101_P015_WB01 UGA_173101_P015_WB03    614    870 FALSE FALSE
## 29 227-23 UGA_173101_P021_WHO6 UGA_173101_P023_WC12   1156   1134 FALSE FALSE
##      colclone1 colclone2 is.clonePLINK is.cloneCOLONY nSNPshared nSNPmatch
## 1           74        118         FALSE         FALSE        519        393
## 29           14        190          TRUE         FALSE       1092        940
##      kinship GTservice mismatch errorRate
## 1  0.309172    LGC        126 0.2427746
## 29 0.424453    LGC        152 0.1391941
```

Inferred as Recaptures?

Ideally, all technical replicates would be identified as such and called recaptures in our analysis. This isn't the case, a couple of the samples don't seem all that similar. Perhaps some contamination?

Removing these samples from the dataset:

- i. 35-SGA - high error rate in replicated sample (24%), not identified as a match by COLONY or PLINK2. Unsure which genotype is accurate so better to drop them both.
- ii. 227-23 - error rate is high again (14%), sample not identified as a match by COLONY. Kinship estimates are high, but the lowest of all pairs of replicated samples that were identified by PLINK as recaptures. Note that this pair of samples showed up in our `sexSamples` markdown too. One replicate is inferred to be male, the other inferred to be female. Best to exclude both samples.

Meanwhile, we should keep the match from PLINK for 935-23 and 151-24 and their replicates. They match at all 122 and 117 shared SNP loci!

```
table(repl.out$is.cloneCOLONY)
```

```
##
## FALSE  TRUE
##      4    39
```

```
table(repl.out$is.clonePLINK)
```

```
##
## FALSE TRUE
##      1    42
```

```
repl.out[which(repl.out$is.cloneCOLONY == FALSE),]
```

```
##      ugaID          vcfID1          vcfID2 nSNPgt1 nSNPgt2 filt1 filt2
## 1 35-SGA UGA_173101_P015_WB01 UGA_173101_P015_WB03      614      870 FALSE FALSE
## 29 227-23 UGA_173101_P021_WH06 UGA_173101_P023_WC12     1156     1134 FALSE FALSE
## 41 935-23          GTS_935-23          GTS_935-23R      123      123 FALSE FALSE
## 53 151-24          GTS_151-24          GTS_151-24R      122      118 FALSE FALSE
##      colclone1 colclone2 is.clonePLINK is.cloneCOLONY nSNPshared nSNPmatch
## 1          74          118          FALSE          FALSE          519          393
## 29          14          190          TRUE          FALSE         1092          940
## 41          270          303          TRUE          FALSE          122          122
## 53          270          33          TRUE          FALSE          117          117
##      kinship GTservice mismatch errorRate
## 1 0.309172      LGC          126 0.2427746
## 29 0.424453      LGC          152 0.1391941
## 41 0.500000      GTSeek          0 0.0000000
## 53 0.500000      GTSeek          0 0.0000000
```

```
repl.out[which(repl.out$is.cloneCOLONY == FALSE & repl.out$is.clonePLINK == FALSE),]
```

```
##      ugaID          vcfID1          vcfID2 nSNPgt1 nSNPgt2 filt1 filt2
## 1 35-SGA UGA_173101_P015_WB01 UGA_173101_P015_WB03      614      870 FALSE FALSE
##      colclone1 colclone2 is.clonePLINK is.cloneCOLONY nSNPshared nSNPmatch
## 1          74          118          FALSE          FALSE          519          393
##      kinship GTservice mismatch errorRate
## 1 0.309172      LGC          126 0.2427746
```

```
#Drop samples by setting FILT to TRUE
which(smd$ugaID == "35-SGA")
```

```
## [1] 13 15
```

```
smd$FILT[which(smd$ugaID == "35-SGA")] <- TRUE
which(smd$ugaID == "227-23")
```

```
## [1] 648 786
```

```
smd$FILT[which(smd$ugaID == "227-23")] <- TRUE
```

Check for Odd Patterns

Beyond the technical replicates, we have some other information that will be useful for evaluating the quality of the inferred recaptures. For instance, some bears were directly captured and have known sex information. We wouldn't want to mistake males for females in these samples. Space is also useful, a CGA bear is unlikely to be sampled in NGA and vice versa.

```

nSex <- nSexGT <- nRegion <- c()
for(i in 1:nrow(col.out)) {
  tmpclone <- strsplit(col.out[i,3], split = ",")[1]
  tmp <- smd[which(smd$vcfID %in% tmpclone),]
  nSex[i] <- length(unique(tmp$knownSEX[tmp$knownSEX != ""]))
  nSexGT[i] <- length(unique(tmp$sexGT[tmp$sexGT != "undet"]))
  nRegion[i] <- length(unique(tmp$region))
  rm(tmp, tmpclone)
}

```

Known Sex

We should not have multiple known sexes per clone

```
nSex
```

```

##      [1] 0 0 1 2 1 0 0 0 0 1 1 1 1 0 2 1 1 1 1 2 1 0 1 1 1 0 1 0 0 1 0 1 1 0 1 0 1
##     [38] 1 1 1 0 0 0 2 2 1 1 1 0 1 1 1 1 1 0 1 0 1 1 1 0 1 0 0 0 1 1 0 2 2 1 2 0 1
##     [75] 1 1 1 0 1 1 1 2 1 1 0 1 1 1 1 1 1 1 1 2 1 0 1 1 1 1 1 1 0 1 1 1 1 1 1 1
##    [112] 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 2 0 0 1 1 1 1 1 1 1 1 0 1
##    [149] 1 1 1 0 1 2 1 1 1 1 0 1 1 0 1 1 1 1 1 0 1 1 0 1 1 0 0 1 0 1 2 0 1 0 1 0 1
##    [186] 1 2 1 1 0 0 1 1 0 0 1 0 1 0 0 1 0 0 0 0 1 0 0 1 0 1 0 0 0 0 0 0 0 0 1 1 0 1
##    [223] 0 0 1 0 0 0 0 0 0 0 0 2 1 0 1 1 1 0 1 0 0 1 1 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1
##    [260] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
##    [297] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
##    [334] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1

```

```
max(nSex)
```

```
## [1] 2
```

```
#Does genotype-inferred sex match known sex of samples?
```

```
table(smd$sexGT[smd$knownSEX == "F"])
```

```

##
##      F undet
##     64      3

```

```
table(smd$sexGT[smd$knownSEX == "M"])
```

```

##
##      F      M undet
##     1     82      4

```

```
#The single case of a mismatch (within a sample) between a known sex
# and genotype-inferred sex
```

```
smd[which(smd$knownSEX=="M" & smd$sexGT == "F"),]
```

```
##          extID ugaID is.SCR is.rep week year snare utmSnare
## 71 UGA_173101_P015_WF11 63-DC FALSE FALSE <NA> 2024 <NA> <NA>
##
##                                     fqfile
## 71 RAPiD-Genomics_F534_UGA_173101_P015_WF11_i5-502_i7-51_S14809_L003_R1_001.fastq
##      knownSEX sampSOURCE region nSNPgt          vcfID  FILTER GTservice sexGT
## 71          M   Den Check    CGA    1198 UGA_173101_P015_WF11 FALSE      LGC    F

bigMD <- read.csv("../input/Finalized Hair Samples w Metadata_Carr.csv", header = TRUE)
bigMD[bigMD$Hair.ID == smd$ugaID[which(smd$knownSEX=="M" & smd$sexGT == "F")],]
```

```
##      Sample.. RandomID Hair.ID   Rapid.Genomics.ID Site.ID Week Check.Date
## 1112      1063      1063   63-DC UGA_173101_P012_WC09
##      Strand Sample.Quality DNR.ID          Database.ID      PIT.Tag      Bear Sex
## 1112                                     CGBB_M_2024_C_87815B 606-306-842 87815 C2    M
##      Year   Date   County Region      What Plate Follicles Abnormalitiy.
## 1112 2024 3/2/24 Houston    CGA Den Check    12      14
##      Comments          Address Latitude Longitude X X.1
## 1112 87815 Biological Cub 2 87815 Den 2024 32.52144 -83.44958
```

```
#This was a captured cub, identified as male in the field
# we saw this individual before - see the `sexSamples` markdown for more detail
```

Genotype-inferred Sex

Similarly, our genotype-inferred sexes should not vary within clones.

```
nSexGT

##      [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1
##      [38] 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
##      [75] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
##      [112] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
##      [149] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
##      [186] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
##      [223] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1
##      [260] 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
##      [297] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
##      [334] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

```
max(nSexGT)
```

```
## [1] 2
```

```
which(nSexGT > 1)
```

```
## [1] 14 30 45 254 270
```

```
#They do in a few cases...
```

```
# 14 30 45 254 270
```

```
#Clone 14 - 32 samples, 28 female, 2 male, 2 undet - no direct capture to confirm majority
```

```
table(smd$sexGT[smd$vcfID %in% strsplit(col.out[col.out$CloneIndex == 14,3], ",")[[1]])
```

```
##
##      F      M undet
##    28      2      2
```

```
smd$knownSEX[smd$vcfID %in% strsplit(col.out[col.out$CloneIndex == 14,3], ",")[[1]]]
```

```
## [1] "" "" "" "" "" "" "" "" "" "" "" "" "" "" "" "" "" "" "" "" "" "" "" "" ""
## [26] "" "" "" "" "" "" "" ""
```

```
#Clone 30 - 6 samples, 5 female, 1 male - direct capture confirms majority
table(smd$sexGT[smd$vcfID %in% strsplit(col.out[col.out$CloneIndex == 30,3], ",")[[1]])
```

```
##
## F M
## 5 1
```

```
smd$knownSEX[smd$vcfID %in% strsplit(col.out[col.out$CloneIndex == 30,3], ",")[[1]]]
```

```
## [1] "F" "" "" "" "" ""
```

```
#Clone 45 - 6 samples, 5 female, 1 male - direct capture confirms majority
table(smd$sexGT[smd$vcfID %in% strsplit(col.out[col.out$CloneIndex == 45,3], ",")[[1]])
```

```
##
## F M
## 5 1
```

```
smd$knownSEX[smd$vcfID %in% strsplit(col.out[col.out$CloneIndex == 45,3], ",")[[1]]]
```

```
## [1] "F" "" "" NA NA NA
```

```
#Clone 254 - 9 samples, 8 male, 1 female - no direct capture to confirm majority
table(smd$sexGT[smd$vcfID %in% strsplit(col.out[col.out$CloneIndex == 254,3], ",")[[1]])
```

```
##
## F M
## 1 8
```

```
smd$knownSEX[smd$vcfID %in% strsplit(col.out[col.out$CloneIndex == 254,3], ",")[[1]]]
```

```
## [1] NA NA NA NA NA NA NA NA NA
```

```
#Clone 270 - 28 samples, 25 male, 1 female, 2 undet - no direct capture to confirm majority
table(smd$sexGT[smd$vcfID %in% strsplit(col.out[col.out$CloneIndex == 270,3], ",")[[1]])
```

```
##
##      F      M undet
##     1     25      2
```

```
smd$knownSEX[smd$vcfID %in% strsplit(col.out[col.out$CloneIndex == 270,3], ",")[[1]]]
```

[illegible]

So sex is not inferred perfectly. That's not surprising, given the pretty lenient threshold we're using to identify males (just 5 reads required at 2 SNPs within 3 Y-specific scaffolds). Contamination could easily lead to us calling a sample a male erroneously. Nonetheless, instances with multiple sexes are relatively rare, and in all cases there is either a clear majority from the set of samples or a direct capture that confirms the consensus from the set.

Region

We shouldn't have multiple regions for any clones...

nRegion

[illegible]

```
which(nRegion > 1)
```

```
## [1] 3 61 296
```

```
smd$region[smd$vcfID %in% strsplit(col.out[col.out$CloneIndex==3,3],",")[[1]]]
```

```
## [1] "CGA"    "CGA"    "CGA"    "CGA"    "CGA"    "CGA"    "CGA"    "CGA"    "CGA"  
## [10] "CGA-H"  "CGA"    "CGA"    "CGA"
```

#Not really a separate region, just a historic hair sample (from 2012)

```
smd$region[smd$vcfID %in% strsplit(col.out[col.out$CloneIndex==61,3],",")[[1]]]
```

```
## [1] "CGA-H" "CGA"
```

#Not really a separate region, just a historic hair sample (from 2012)

```
smd$region[smd$vcfID %in% strsplit(col.out[col.out$CloneIndex==296,3],",")[[1]]]
```

```
## [1] "CGA" "CGA" "CGA" "CGA" "CGA" "CGA" "CGA" "CGA" "CGA" "CGA-H"
```

```
#Not really a separate region, just a historic hair sample (from 2012)
```

So no instances of recaptures identified across regions. That's good!

Known vs. Genotype-inferred Sex

Finally, comparing the known sex of unique genotypes (from direct captures) with that inferred for all samples within a clone...

```
#check for discrepancies between known and genotyped sex for all clones
discrepancy <- multknown <- c()
#for(i in 1:max(sampleMetadata$cloneID, na.rm = TRUE)) {
for(i in 1:nrow(col.out)) {
  tmpclone <- strsplit(col.out[i,3], split = ",")[[1]]
  tmp <- smd[which(smd$vcfID %in% tmpclone),]
  tmp.knownSEX <- unique(tmp$knownSEX[tmp$knownSEX != "" & !is.na(tmp$knownSEX)])
  if(length(tmp.knownSEX) > 0) {
    if(length(tmp.knownSEX) > 1) {
      multknown[i] <- TRUE
      discrepancy[i] <- NA
    } else {
      multknown[i] <- FALSE
      if(sum(tmp$sexGT[tmp$sexGT != "undet"] != tmp.knownSEX) == 0) {
        discrepancy[i] <- FALSE
      } else {
        discrepancy[i] <- TRUE
      }
    }
  }
  rm(tmpclone, tmp, tmp.knownSEX)
}

sum(discrepancy, na.rm = TRUE) #3 cases with discrepancies...
```

```
## [1] 3
```

```
sum(multknown, na.rm = TRUE) #0 cases with multiple "known" sexes
```

```
## [1] 0
```

```
#Check the discrepancies...
which(discrepancy == TRUE)
```

```
## [1] 30 45 144
```

```
#Clone 30 - seen above, capture agrees with majority of samples
```

```
table(smd$knownSEX[smd$vcfID %in% strsplit(col.out[which(col.out$CloneIndex == 30),3],",")[[1]])
```

```
##
```

```
## F
```

```
## 5 1
```

```

#Clone 45 - seen above, capture agrees with majority of samples
table(smd$knownSEX[smd$vcfID %in% strsplit(col.out[which(col.out$CloneIndex == 45),3],",")[[1]])

##
##    F
## 2 1

#Clone 144 - noted previously, cub recorded as M in the field, genotype looks F
smd$ugaID[smd$vcfID %in% strsplit(col.out[which(col.out$CloneIndex == 144),3],",")[[1]]]

## [1] "63-DC"

```

Compare Inferred Recaptures

Now we create a new data frame to compare inferences (same individual / not) for each pair of samples. Note that, for PLINK, we're using 0.354 as a threshold for KING relatedness coefficients of duplicated samples. This is based off of information provided in the [PLINK2 manual](#). Quoting directly (emphasis added)...

Note that KING kinship coefficients are scaled such that duplicate samples have kinship 0.5, not 1. First-degree relations (parent-child, full siblings) correspond to ~0.25, second-degree relations correspond to ~0.125, etc. It is conventional to use a cutoff of ~0.354 (the geometric mean of 0.5 and 0.25) to screen for monozygotic twins and duplicate samples, ~0.177 to add first-degree relations, etc.

However, based on our replicated genotypes, we will increase this threshold when combining inferences from PLINK and COLONY (to a mean kinship - across samples within clones - of at least 0.45).

Overall summaries of the matches

First let's see how many pairs of samples are identified as the same individual in our analyses...

```

#Create a new data frame with info for each pair...
compare.out <- data.frame(id1 = kingrel$id1,
                          id2 = kingrel$id2,
                          kinship = kingrel$kinship,
                          nsnp = kingrel$nsnp,
                          colclone1 = NA,
                          colclone2 = NA,
                          isclonePLINK = NA,
                          iscloneCOL = NA)

#This loop takes a bit of time
T1 <- Sys.time()
for(r in 1:nrow(kingrel)) {
  tmp.grep1 <- grep(compare.out$id1[r], col.out[,3])
  if(length(tmp.grep1) > 1) {
    for(i in 1:length(tmp.grep1)) {
      tmp.split <- strsplit(col.out[tmp.grep1[i],3], ",")[[1]]
      if(compare.out$id1[r] %in% tmp.split) {
        compare.out$colclone1[r] <- col.out$CloneIndex[tmp.grep1[i]]
      }
      rm(tmp.split)
    }
  }
}

```

```

} else {
  compare.out$colclone1[r] <- col.out$CloneIndex[tmp.grep1]
}

tmp.grep2 <- grep(compare.out$id2[r], col.out[,3])
if(length(tmp.grep2) > 1) {
  for(i in 1:length(tmp.grep2)) {
    tmp.split <- strsplit(col.out[tmp.grep2[i],3], ",")[[1]]
    if(compare.out$id2[r] %in% tmp.split) {
      compare.out$colclone2[r] <- col.out$CloneIndex[tmp.grep2[i]]
    }
    rm(tmp.split)
  }
} else {
  compare.out$colclone2[r] <- col.out$CloneIndex[tmp.grep2]
}

rm(tmp.grep1, tmp.grep2)

if(compare.out$colclone1[r] == compare.out$colclone2[r]) {
  compare.out$iscloneCOL[r] <- TRUE
} else {
  compare.out$iscloneCOL[r] <- FALSE
}

if(!is.na(compare.out$kinship[r])) {
  if(compare.out$kinship[r] >= 0.354) {
    compare.out$isclonePLINK[r] <- TRUE
  } else {
    compare.out$isclonePLINK[r] <- FALSE
  }
} else {
  compare.out$isclonePLINK[r] <- FALSE
}

}
Sys.time()-T1

```

```
## Time difference of 3.565906 hours
```

```

#How many pairs identified by COLONY are also identified by PLINK?
table(compare.out$isclonePLINK[compare.out$iscloneCOL == TRUE])

```

```

##
## TRUE
## 16078

```

```

#How many pairs identified by PLINK are also identified by COLONY?
table(compare.out$iscloneCOL[compare.out$isclonePLINK == TRUE])

```

```

##
## FALSE TRUE
## 19803 16078

```

From a total of 1408681 pairwise comparisons, COLONY identifies 16078 matches, while PLINK identifies 35881 matches. There are 19803 comparisons where PLINK finds a match but COLONY does not and 0 comparisons where COLONY finds a match but PLINK does not. In other words 100% of COLONY matches are confirmed by PLINK, while 44.8% of PLINK matches are confirmed by COLONY.

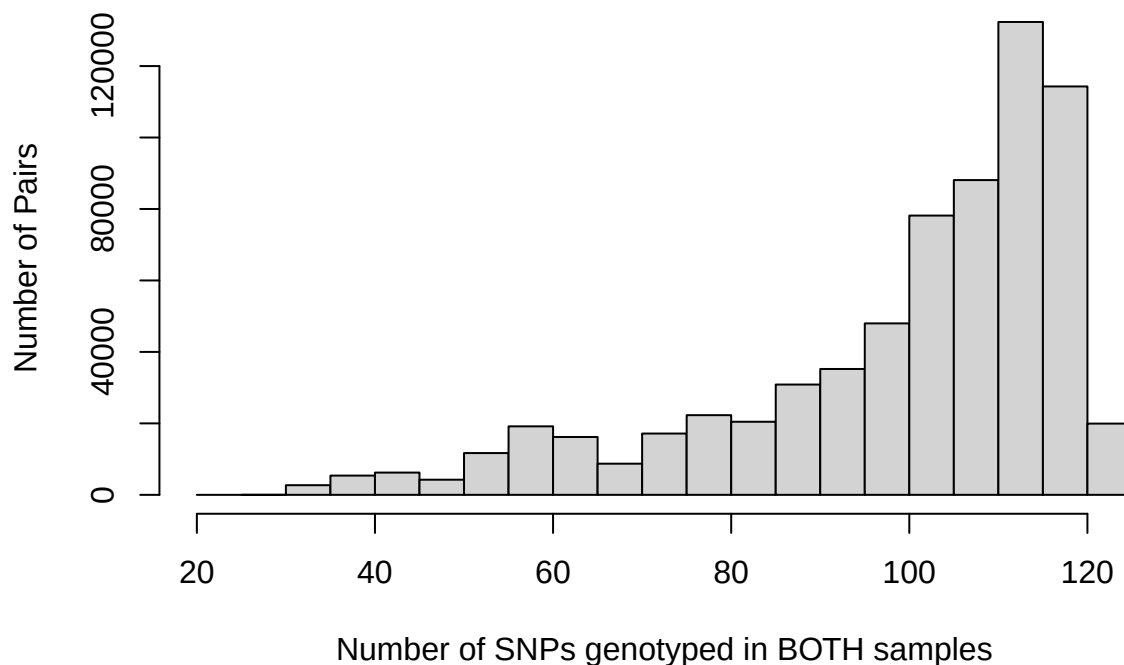
Let's look more closely at some of the discrepancies between the two analyses. First those from the COLONY analysis *only*.

```
compare.out[which(compare.out$iscloneCOL == TRUE & compare.out$isclonePLINK == FALSE),]
```

```
## [1] id1          id2          kinship      nsnp          colclone1
## [6] colclone2     isclonePLINK iscloneCOL
## <0 rows> (or 0-length row.names)
```

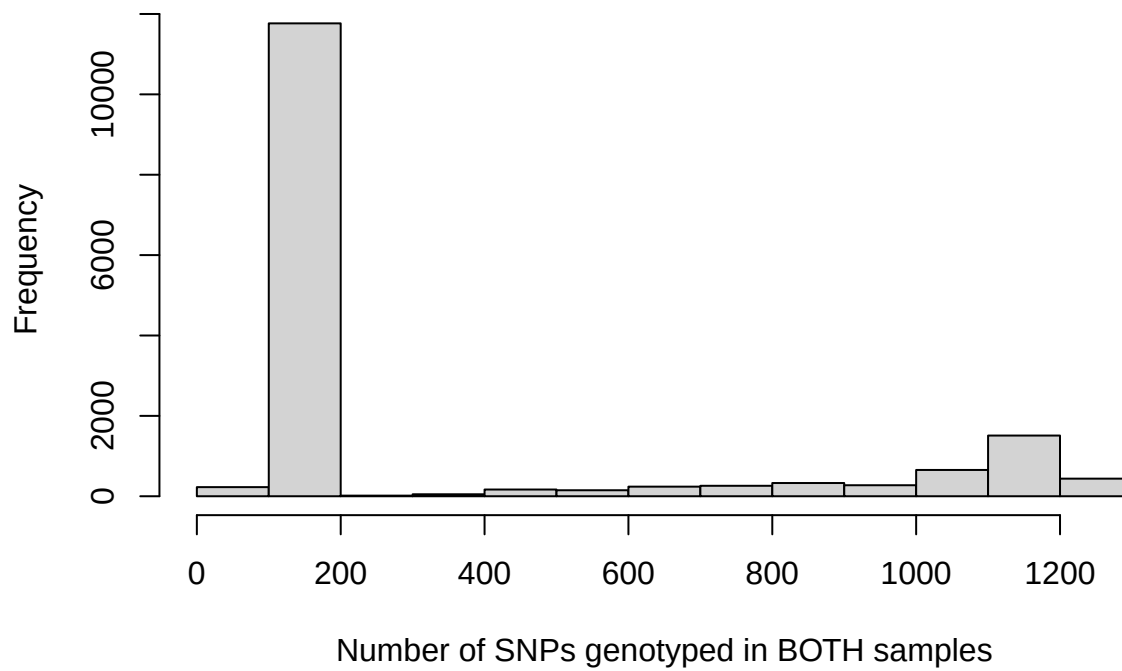
```
hist(compare.out$nsnp[which(compare.out$id1 %in% smd$vcfID[which(smd$GTservice == "GTSeek")] &
  compare.out$id2 %in% smd$vcfID[which(smd$GTservice == "LGC")]) |
  compare.out$id1 %in% smd$vcfID[which(smd$GTservice == "LGC")] &
  compare.out$id2 %in% smd$vcfID[which(smd$GTservice == "GTSeek")])],
  main = "Overlap in SNP loci for comparisons between LGC and GTSeek samples",
  xlab = "Number of SNPs genotyped in BOTH samples", ylab = "Number of Pairs")
```

Overlap in SNP loci for comparisons between LGC and GTSeek samp



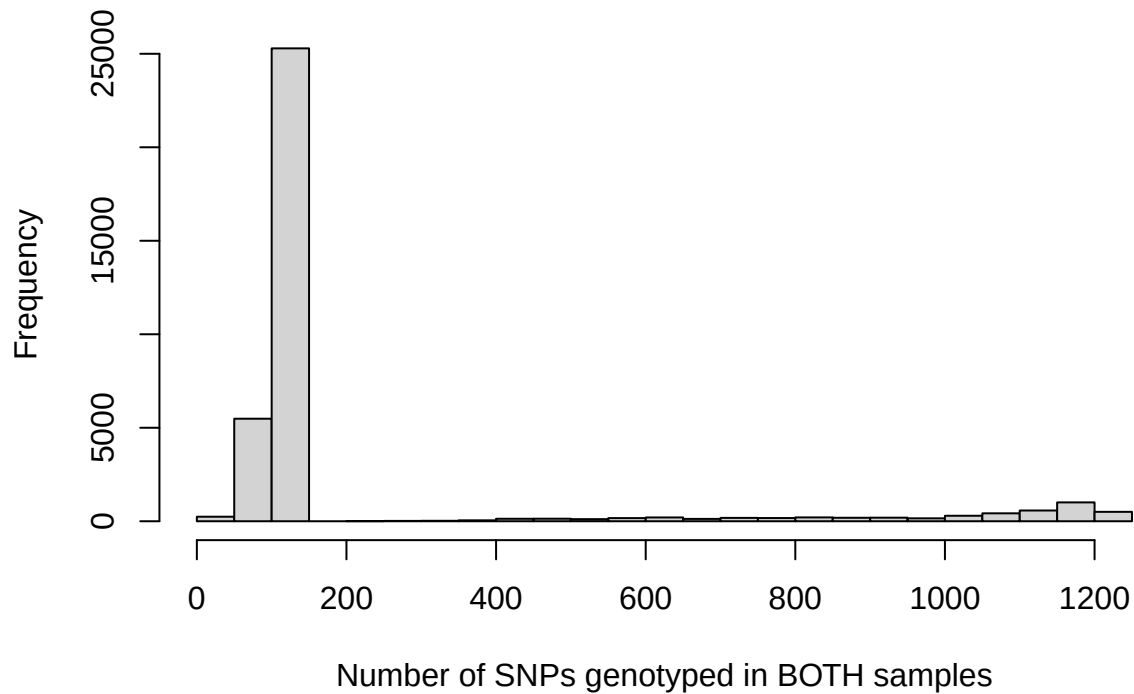
```
hist(compare.out$nsnp[which(compare.out$iscloneCOL == TRUE)],
  main = "Number of SNP loci shared - COLONY matches",
  xlab = "Number of SNPs genotyped in BOTH samples")
```

Number of SNP loci shared – COLONY matches



```
hist(compare.out$nsnp[which(compare.out$isclonePLINK == TRUE)],  
      main = "Number of SNP loci shared - PLINK matches",  
      xlab = "Number of SNPs genotyped in BOTH samples")
```

Number of SNP loci shared – PLINK matches



```
#Range of shared SNPs for COLONY matches b/w GTSeek and LGC
colMatches <- compare.out[which(compare.out$iscloneCOL==TRUE),]
range(colMatches$nsnp[which(colMatches$id1 %in% smd$vcfID[which(smd$GTservice == "GTSeek")] &
colMatches$id2 %in% smd$vcfID[which(smd$GTservice == "LGC")] |
colMatches$id1 %in% smd$vcfID[which(smd$GTservice == "LGC")] &
colMatches$id2 %in% smd$vcfID[which(smd$GTservice == "GTSeek"])]))
```

```
## [1] 39 122
```

```
#Same for PLINK
plinkMatches <- compare.out[which(compare.out$isclonePLINK == TRUE),]
range(plinkMatches$nsnp[which(plinkMatches$id1 %in% smd$vcfID[which(smd$GTservice == "GTSeek")] &
plinkMatches$id2 %in% smd$vcfID[which(smd$GTservice == "LGC")] |
plinkMatches$id1 %in% smd$vcfID[which(smd$GTservice == "LGC")] &
plinkMatches$id2 %in% smd$vcfID[which(smd$GTservice == "GTSeek"])]))
```

```
## [1] 33 125
```

The filters for LGC samples (at least 30% of SNPs genotyped) keep us from matching samples based on *very* few loci.

Final Recapture Calls

Now to combine the output from the two programs using the scheme described above. COLONY must be missing a few genotype matches. For each COLONY clone, are there samples that would have been combined

by PLINK? If so, are *all* members of those clones also identified by PLINK as recaptures of the focal group? What is the mean kinship coefficient estimated between the samples associated with the two COLONY clones?

Below, we set two thresholds for re-assigning a sample to a different COLONY clone. These include:

- i. Mean kinship to samples in the recipient clone ≥ 0.45 (based on range of kinship estimates for technical replicates)
- ii. At least one of the comparisons to samples in the other clone needs to be based on a minimum of 50 SNP loci (this number will show up again below)

Now to reassign samples to clones...

```
## Set up the object
totClones <- max(c(compare.out$colclone1, compare.out$colclone2))
finalCloneDF <- data.frame(initial = c(1:totClones),
                           init.size = NA,
                           prob = NA,
                           final = c(1:totClones),
                           possibles = NA,
                           is.moved = FALSE,
                           is.recip = FALSE)

## Fill in init.size & prob
for(i in 1:totClones) {
  tmpclone <- strsplit(col.out[which(col.out$CloneIndex == i),3], ",")[[1]]
  finalCloneDF$init.size[i] <- length(tmpclone)
  finalCloneDF$prob[i] <- col.out$Prob[which(col.out$CloneIndex == i)]
  rm(tmpclone)
}

## Loop over the clones and identify all possible matches
for(i in 1:totClones) {
  tmp <- compare.out[which(compare.out$colclone1 == i & compare.out$colclone2 != i |
                           compare.out$colclone2 == i & compare.out$colclone1 != i),]
  ## If no other clones are a possible match for the focal clone
  ## set the final clone ID, it doesn't move, and it's not a recipient
  if(sum(tmp$isclonePLINK) == 0) {
    finalCloneDF$final[i] <- i
  } else {
    ## If some clones do match...
    ## identify the set of clones that match the focal clone
    tmp.match <- unique(c(tmp$colclone1[tmp$isclonePLINK==TRUE],
                          tmp$colclone2[tmp$isclonePLINK==TRUE]))
    tmp.match <- tmp.match[-which(tmp.match == i)]
    ## then continue adding possible matches to all of these other clones
    ## do this until the list of possibilities stops growing!!
    growing <- TRUE
    while(growing) {
      tmp.tmp.match <- unique(
        c(compare.out$colclone1[which(compare.out$colclone1 %in% tmp.match &
                                      compare.out$colclone2 %in% tmp.match &
                                      compare.out$iscloneCOL == FALSE &
                                      compare.out$isclonePLINK == TRUE)],
          compare.out$colclone2[which(compare.out$colclone1 %in% tmp.match &
```

```

compare.out$colclone2 %in% tmp.match &
compare.out$iscloneCOL == FALSE &
compare.out$isclonePLINK == TRUE)))]))
tmp.tmp.match <- tmp.tmp.match[which(tmp.tmp.match == i)]
if(length(tmp.tmp.match) > length(tmp.match)) {
  tmp.match <- tmp.tmp.match
} else {
  growing <- FALSE
}
rm(tmp.tmp.match)
}

#Record the possible matches...
finalCloneDF$possibles[i] <- paste0(tmp.match, collapse="_")

## If any of these clones have been moved to a new clone...
## replace them in the tmp.match list with their new clone
for(j in tmp.match) {
  if(finalCloneDF$final[which(finalCloneDF$initial == j)] != j) {
    tmp.match[which(tmp.match == j)] <- finalCloneDF$final[which(finalCloneDF$initial == j)]
  }
}
tmp.match <- unique(tmp.match[which(tmp.match != i)])
## This means we'll need to refer to the final column when calculating
## mean kinship values for the pairs below...

## Now check each of these to ask if it should be moved...
real.matches <- i #The focal clone, which can not have moved yet...

for(j in tmp.match) {
  i.set <- unique(c(
    real.matches, #The focal clone plus any earlier in the list that matched
    #Other clones previously moved into the focal clone
    finalCloneDF$initial[which(finalCloneDF$final == i)]
  ))
  j.set <- unique(c(
    j, #The possible match
    #If j has moved, it's final clone
    finalCloneDF$final[which(finalCloneDF$initial == j)],
    #Other clones that previously moved into j's final clone
    finalCloneDF$initial[which(finalCloneDF$final ==
      finalCloneDF$final[which(finalCloneDF$initial ==
        j)])]
  ))

  subtmp <- compare.out[which(compare.out$colclone1 %in% i.set &
    compare.out$colclone2 %in% j.set |
    compare.out$colclone1 %in% j.set &
    compare.out$colclone2 %in% i.set),]

  ## Focus on comparisons involving the possible mover - j
  #subtmp <- subtmp[which(subtmp$colclone1 == j | subtmp$colclone2 == j),]

```

```

## Check mean kinship
if(mean(subtmp$kinship) >= 0.45 & max(subtmp$nsnp) >= 50) {
  real.matches <- c(real.matches, j)
}
rm(subtmp, i.set, j.set)
}

## Now if there are real matches, figure out where to put the set of samples
if(length(real.matches) > 1) {
  tmp.sizes <- data.frame(finalClone=unique(finalCloneDF$final[real.matches]),
                          finalSize = NA)
  for(r in 1:nrow(tmp.sizes)) {
    tmp.sizes$finalSize[r] <- sum(
      finalCloneDF$init.size[which(finalCloneDF$final ==
                                   tmp.sizes$finalClone[r])]
    )
  }
  bigclone <- tmp.sizes$finalClone[which(tmp.sizes$finalSize ==
                                         max(tmp.sizes$finalSize))]

  if(length(bigclone) > 1) {
    bigclone <- min(bigclone)
  }

  finalCloneDF$final[real.matches] <- bigclone

  rm(tmp.sizes, bigclone)
}
rm(tmp.match)
}

#Did clone move? Was it a recipient of movers?
for(i in 1:totClones) {
  if(finalCloneDF$initial[i] != finalCloneDF$final[i]) {
    finalCloneDF$is.moved[i] <- TRUE
  }
  if(length(which(finalCloneDF$final==i)) > 1) {
    finalCloneDF$is.recip[i] <- TRUE
  }
}

#Structure of the object
head(finalCloneDF)

```

```

##   initial init.size  prob final      possibles is.moved
## 1      1         6 1.000     1          259     FALSE
## 2      2         1 0.221    262    32_207_226_240_258_262_274     TRUE
## 3      3        13 0.998     3          201_251     FALSE
## 4      4        43 0.983     4 6_65_68_213_229_236_250_256_278     FALSE
## 5      5         3 1.000     5          321     FALSE
## 6      6         1 1.000     6      4_68_162_203_221_256_278     FALSE
##   is.recip
## 1      TRUE

```

```
## 2    FALSE
## 3     TRUE
## 4     TRUE
## 5    FALSE
## 6    FALSE
```

```
#How many clones total from COLONY
nrow(finalCloneDF)
```

```
## [1] 370
```

```
#How many survive?
length(unique(finalCloneDF$final))
```

```
## [1] 313
```

```
#Difference
length(unique(finalCloneDF$final)) - nrow(finalCloneDF)
```

```
## [1] -57
```

```
#How many moved?
sum(finalCloneDF$initial != finalCloneDF$final)
```

```
## [1] 57
```

```
sum(finalCloneDF$is.moved)
```

```
## [1] 57
```

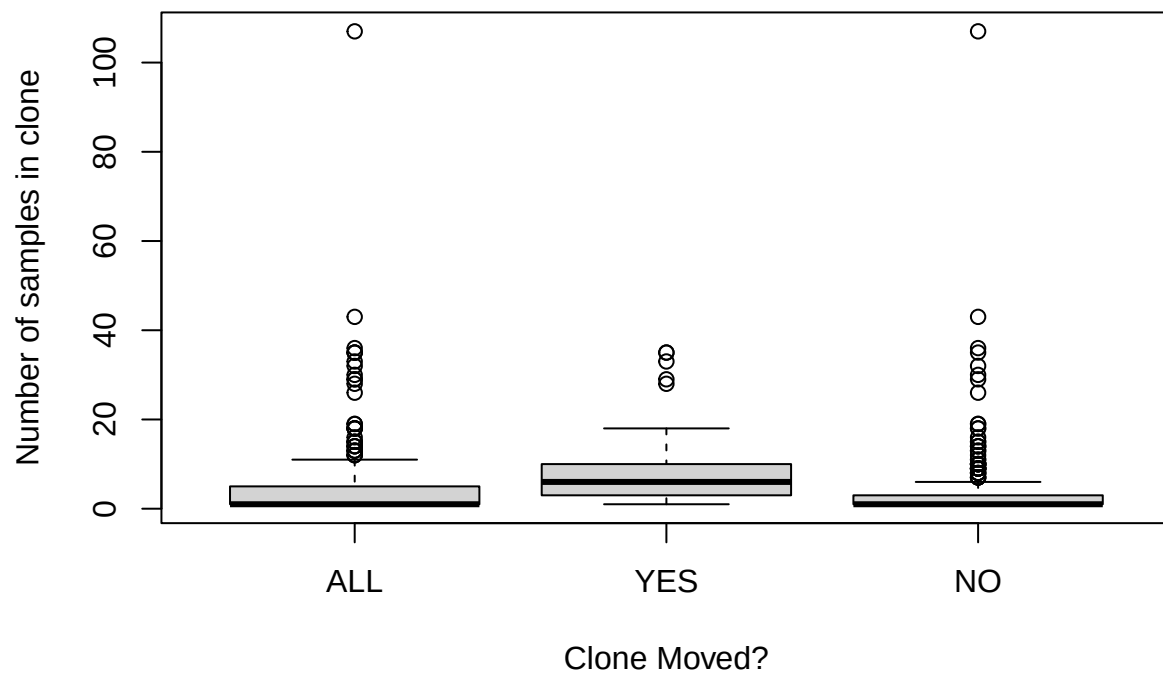
```
#Into how many recipients?
sum(finalCloneDF$is.recip)
```

```
## [1] 37
```

```
#Shouldn't be any recipients that were moved
table(finalCloneDF$is.moved[which(finalCloneDF$is.recip == TRUE)])
```

```
##
## FALSE
##    37
```

```
#Did clones reassigned to other clones tend to be sparse?
boxplot(finalCloneDF$init.size,
        finalCloneDF$init.size[finalCloneDF$is.moved],
        finalCloneDF$init.size[!finalCloneDF$is.moved],
        names = c("ALL", "YES", "NO"),
        xlab = "Clone Moved?",
        ylab = "Number of samples in clone")
```



#It doesn't appear that they were

#Did clones that were moved tend to have lower probabilities from COLONY?

```
boxplot(finalCloneDF$prob,
        finalCloneDF$prob[finalCloneDF$is.moved],
        finalCloneDF$prob[!finalCloneDF$is.moved],
        names = c("ALL", "YES", "NO"),
        xlab = "Clone Moved?",
        ylab = 'Clone "Probability" (from COLONY)')
```



#Yes, COLONY tended to be less confident about clones that were reassigned...

Interested to look more closely at clones that were moved by this combination. In those cases, there should be more than one row of the dataset (initial COLONY clone) that gets assigned to a particular final clone.

```
#Clones that were the recipients of other clones
table(finalCloneDF$final)
```

```
##
##  1  3  4  5  6  7  8 11 14 15 17 18 19 20 21 22 24 29 33 34
##  2  2  5  1  1  3  2  1  3  1  1  1  1  1  1  1  1  1  5  1
## 35 37 38 40 41 42 43 44 45 47 48 49 50 51 52 53 54 56 57 58
##  1  1  1  1  1  1  2  1  1  1  1  1  1  1  1  1  1  1  1  1
## 59 60 62 64 66 67 68 69 70 71 72 73 74 75 76 77 79 80 81 82
##  1  1  1  1  1  1  1  2  1  1  1  1  1  1  1  1  1  1  1  1
## 83 84 85 86 87 88 89 90 91 92 94 95 96 98 99 101 102 103 105 106
##  1  2  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1
## 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
##  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1
## 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145 146
##  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1
## 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162 163 164 165 166
##  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1
## 167 168 169 170 171 172 173 174 176 177 179 180 181 183 184 185 186 187 188 190
##  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1
```

```
## 191 192 193 194 195 196 197 198 199 202 203 204 205 206 207 208 209 210 211 212
## 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 230 231 232 233
## 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## 234 235 236 237 238 239 241 242 243 244 246 247 248 249 250 251 252 253 255 257
## 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2
## 260 261 262 263 265 266 269 271 272 275 277 280 281 282 283 284 286 287 288 291
## 1 2 6 1 2 1 2 2 4 2 2 2 2 2 2 2 2 1 4 3
## 292 293 294 297 298 299 300 301 302 304 306 307 308 309 310 311 312 313 316 317
## 3 1 1 2 3 2 1 1 1 3 1 2 1 2 1 1 1 1 2 1
## 318 319 320 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337
## 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## 338 339 340 341 342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357
## 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## 358 359 360 361 362 363 364 365 366 367 368 369 370
## 1 1 1 1 1 1 1 1 1 1 1 1 1
```

```
sum(table(finalCloneDF$final) > 1)
```

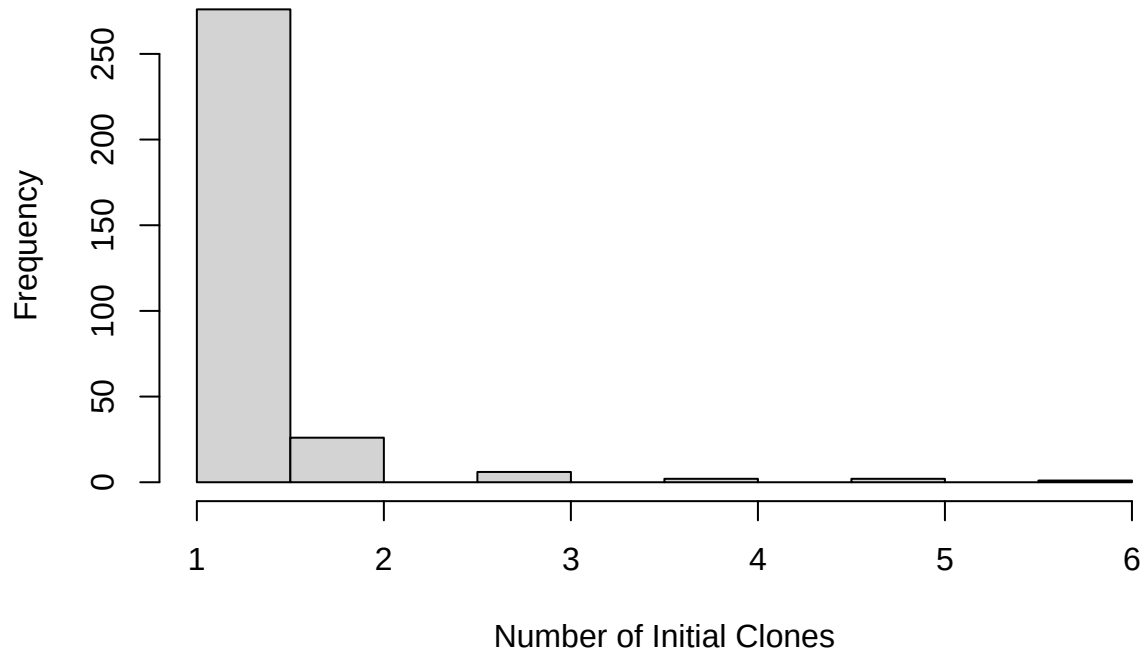
```
## [1] 37
```

```
names(which(table(finalCloneDF$final) > 1))
```

```
## [1] "1" "3" "4" "7" "8" "14" "33" "43" "69" "84" "255" "257"
## [13] "261" "262" "265" "269" "271" "272" "275" "277" "280" "281" "282" "283"
## [25] "284" "286" "288" "291" "292" "297" "298" "299" "304" "307" "309" "316"
## [37] "322"
```

```
hist(table(finalCloneDF$final), main = "Final Clone Membership",
      xlab = "Number of Initial Clones")
```

Final Clone Membership



#Spot checking moved clones... top 3 recipients

```
spotClones <- names(sort(table(finalCloneDF$final), decreasing = TRUE))[1:3]
```

```
cloneInQ <- spotClones[1]
```

```
finalCloneDF[which(finalCloneDF$final == cloneInQ),]
```

##	initial	init.size	prob	final	possibles	is.moved	is.recip
## 2	2	1	0.221	262	32_207_226_240_258_262_274	TRUE	FALSE
## 32	32	35	0.356	262	207_226_240_258_262_274_2	TRUE	FALSE
## 240	240	1	0.451	262	32_258_262_274_2_207_226	TRUE	FALSE
## 258	258	2	0.356	262	262_274_2_32_207_226_240	TRUE	FALSE
## 262	262	36	0.356	262	274_258_2_32_207_226_240	FALSE	TRUE
## 274	274	18	0.356	262	262_258_2_32_207_226_240	TRUE	FALSE

```
finalCloneDF[finalCloneDF$initial %in%
```

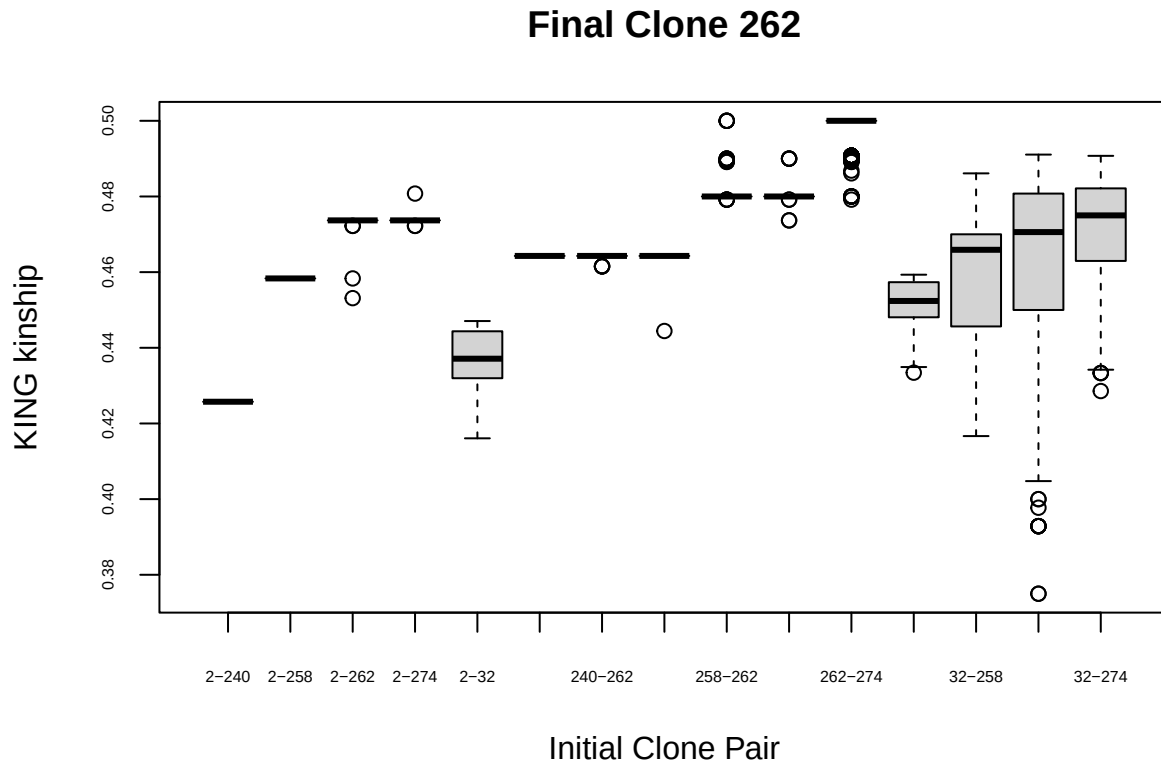
```
  unique(unlist(strsplit(
```

```
    finalCloneDF$possibles[finalCloneDF$final == cloneInQ], "_"))],]
```

##	initial	init.size	prob	final	possibles	is.moved	is.recip
## 2	2	1	0.221	262	32_207_226_240_258_262_274	TRUE	FALSE
## 32	32	35	0.356	262	207_226_240_258_262_274_2	TRUE	FALSE
## 207	207	1	0.988	207	32_240_258_262_274_2	FALSE	FALSE
## 226	226	1	0.495	226	240_32_258_262_274_2	FALSE	FALSE
## 240	240	1	0.451	262	32_258_262_274_2_207_226	TRUE	FALSE
## 258	258	2	0.356	262	262_274_2_32_207_226_240	TRUE	FALSE

## 262	262	36	0.356	262	274_258_2_32_207_226_240	FALSE	TRUE
## 274	274	18	0.356	262	262_258_2_32_207_226_240	TRUE	FALSE

```
tmp.comp <- compare.out[which(
  compare.out$colclone1 %in%
    finalCloneDF$initial[which(finalCloneDF$final == cloneInQ)] &
  compare.out$colclone2 %in%
    finalCloneDF$initial[which(finalCloneDF$final == cloneInQ)] &
  compare.out$colclone1 != compare.out$colclone2),]
boxlabs <- apply(tmp.comp[,c("colclone1","colclone2")], 1,
  FUN=function(x) paste(sort(x), collapse = "-"))
boxplot(tmp.comp$kinship ~ boxlabs,
  ylab = "KING kinship", xlab = "Initial Clone Pair",
  main = paste("Final Clone",cloneInQ), cex.axis = 0.5)
```



```
cloneInQ <- spotClones[2]
finalCloneDF[which(finalCloneDF$final == cloneInQ),]
```

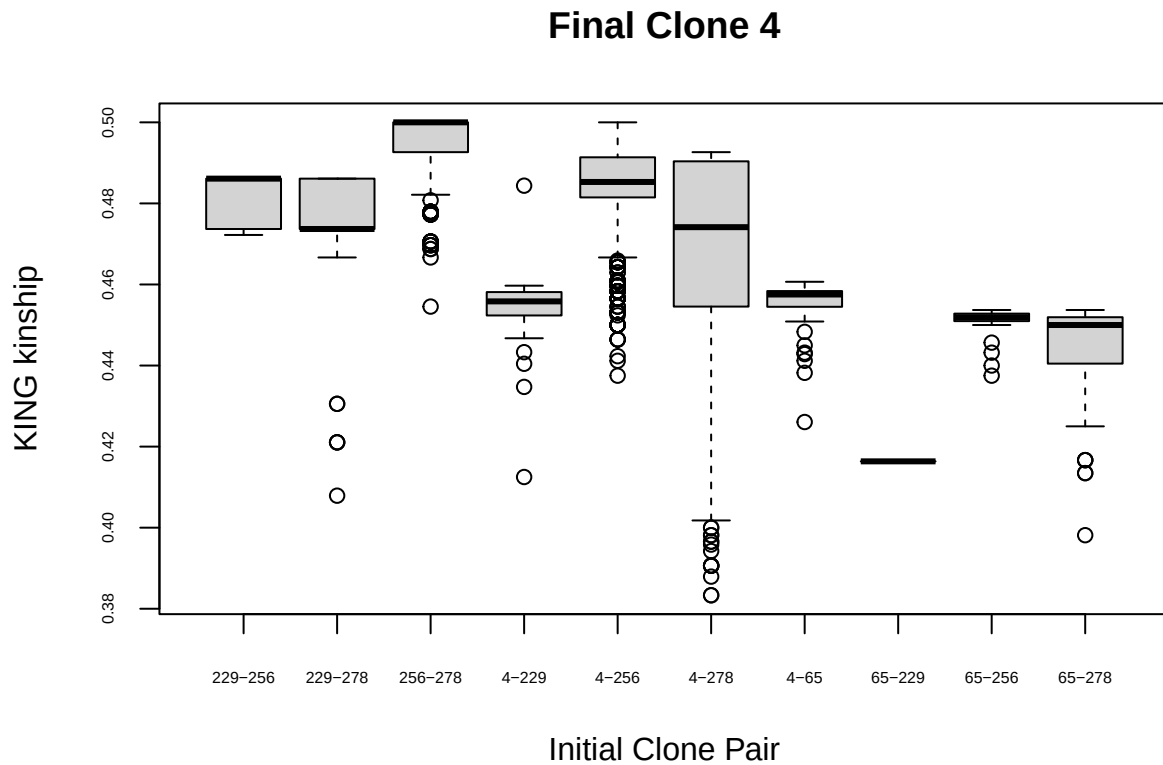
##	initial	init.size	prob	final	possibles	is.moved
## 4	4	43	0.983	4	6_65_68_213_229_236_250_256_278	FALSE
## 65	65	1	0.571	4	4_68_213_229_236_250_256_278	TRUE
## 229	229	1	0.260	4	236_4_250_256_278_65_213	TRUE
## 256	256	35	0.471	4	278_4_6_65_68_213_229_236	TRUE
## 278	278	29	0.471	4	4_256_65_68_213_229_236_6_250	TRUE
##	is.recip					

```
## 4      TRUE
## 65     FALSE
## 229    FALSE
## 256    FALSE
## 278    FALSE
```

```
finalCloneDF[finalCloneDF$initial %in%
  unique(unlist(strsplit(
    finalCloneDF$possibles[finalCloneDF$final == cloneInQ], "_"))),]
```

##	initial	init.size	prob	final	possibles	is.moved
## 4	4	43	0.983	4	6_65_68_213_229_236_250_256_278	FALSE
## 6	6	1	1.000	6	4_68_162_203_221_256_278	FALSE
## 65	65	1	0.571	4	4_68_213_229_236_250_256_278	TRUE
## 68	68	1	1.000	68	4_162_221_256_278_6_65	FALSE
## 213	213	1	0.611	213	4_229_236_250_256_278_65	FALSE
## 229	229	1	0.260	4	236_4_250_256_278_65_213	TRUE
## 236	236	1	0.451	236	4_256_278_65_213_229	FALSE
## 250	250	1	0.811	250	278_4_65_213_229	FALSE
## 256	256	35	0.471	4	278_4_6_65_68_213_229_236	TRUE
## 278	278	29	0.471	4	4_256_65_68_213_229_236_6_250	TRUE
##	is.recip					
## 4	TRUE					
## 6	FALSE					
## 65	FALSE					
## 68	FALSE					
## 213	FALSE					
## 229	FALSE					
## 236	FALSE					
## 250	FALSE					
## 256	FALSE					
## 278	FALSE					

```
tmp.comp <- compare.out[which(
  compare.out$colclone1 %in%
    finalCloneDF$initial[which(finalCloneDF$final == cloneInQ)] &
  compare.out$colclone2 %in%
    finalCloneDF$initial[which(finalCloneDF$final == cloneInQ)] &
  compare.out$colclone1 != compare.out$colclone2),]
boxlabs <- apply(tmp.comp[,c("colclone1", "colclone2")], 1,
  FUN=function(x) paste(sort(x), collapse = "-"))
boxplot(tmp.comp$kinship ~ boxlabs,
  ylab = "KING kinship", xlab = "Initial Clone Pair",
  main = paste("Final Clone", cloneInQ), cex.axis = 0.5)
```



```
cloneInQ <- spotClones[3]
finalCloneDF[which(finalCloneDF$final == cloneInQ),]
```

##	initial	init.size	prob	final	possibles	is.moved	is.recip
## 33	33	29	0.171	33	270_290_303_315	FALSE	TRUE
## 270	270	28	0.171	33	290_33_303_315	TRUE	FALSE
## 290	290	5	0.171	33	270_33_303_315	TRUE	FALSE
## 303	303	4	0.171	33	33_290_315_270	TRUE	FALSE
## 315	315	2	0.171	33	270_303_290_33	TRUE	FALSE

```
finalCloneDF[finalCloneDF$initial %in%
  unique(unlist(strsplit(
    finalCloneDF$possibles[finalCloneDF$final == cloneInQ], "_"))),]
```

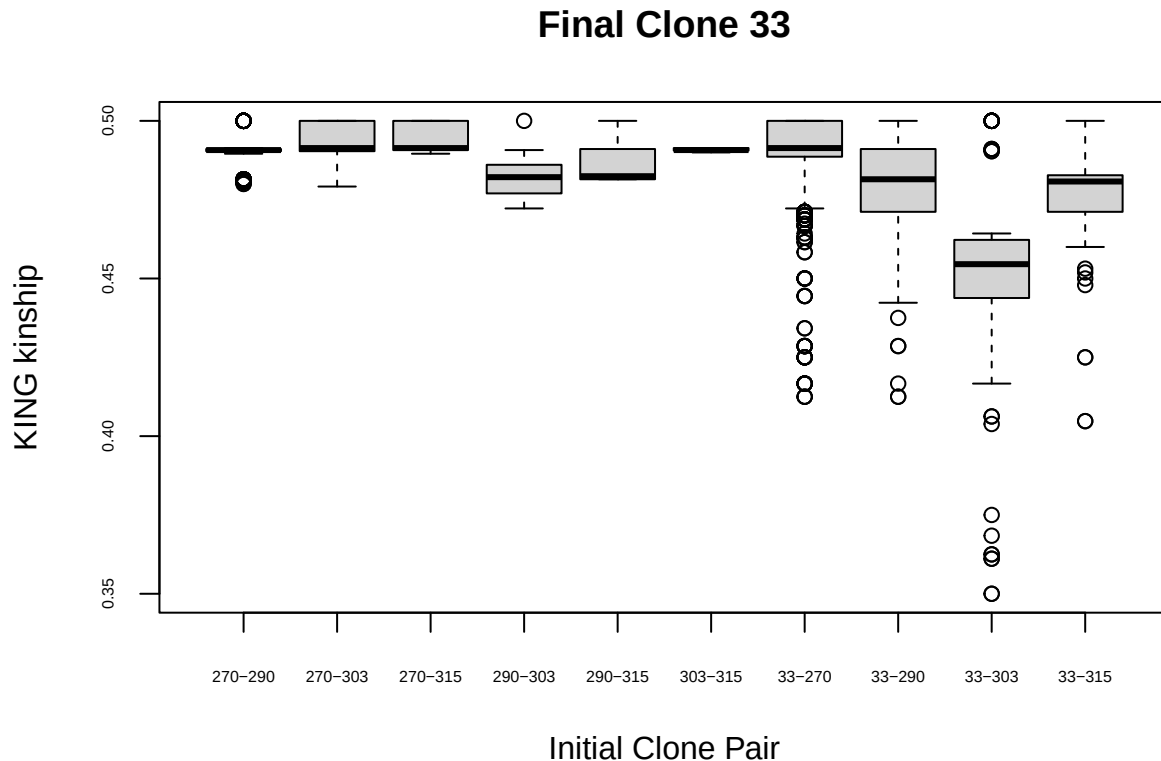
##	initial	init.size	prob	final	possibles	is.moved	is.recip
## 33	33	29	0.171	33	270_290_303_315	FALSE	TRUE
## 270	270	28	0.171	33	290_33_303_315	TRUE	FALSE
## 290	290	5	0.171	33	270_33_303_315	TRUE	FALSE
## 303	303	4	0.171	33	33_290_315_270	TRUE	FALSE
## 315	315	2	0.171	33	270_303_290_33	TRUE	FALSE

```
tmp.comp <- compare.out[which(
  compare.out$colclone1 %in%
    finalCloneDF$initial[which(finalCloneDF$final == cloneInQ)] &
```

```

compare.out$colclone2 %in%
finalCloneDF$initial[which(finalCloneDF$final == cloneInQ)] &
compare.out$colclone1 != compare.out$colclone2),]
boxlabs <- apply(tmp.comp[,c("colclone1", "colclone2")], 1,
  FUN=function(x) paste(sort(x), collapse = "-"))
boxplot(tmp.comp$kinship ~ boxlabs,
  ylab = "KING kinship", xlab = "Initial Clone Pair",
  main = paste("Final Clone", cloneInQ), cex.axis = 0.5)

```



Overall this looks pretty reasonable. There were some low-kinship outliers within some of these clones, though.

Another check, do we ever move samples into a clone that has already been removed from the population? In other words, clone B is identified as a recapture of clone A and moved accordingly. Subsequently, clone C is identified as a recapture of clone B (but not clone A) and moved into an “empty” clone. Does this happen?

```

#Any cases where a clone gets emptied out (moved to another, larger clone)
# but then another clone is assigned to the emptied clone?

#This should show up as rows where initial != final, but is.recip = TRUE
which(finalCloneDF$is.recip == TRUE &
  finalCloneDF$initial != finalCloneDF$final)

```

```
## integer(0)
```

Check for Odd Patterns in the FINAL clone assignments

Want to revisit the checks above, given our new clone assignments.

```
#Add COLclone to smd
smd$COLclone <- NA
smd$finalClone <- NA
for(i in 1:nrow(smd)){
  thisclone <- col.out$CloneIndex[grep(smd$vcfID[i], col.out[,3])]

  if(length(thisclone) > 1) {
    this.this <- which(sapply(strsplit(col.out[thisclone,3],","), FUN=function(x) sum(x==smd$vcfID[i])))
    thisclone <- thisclone[this.this]
    rm(this.this)
  }

  if(length(thisclone) > 0) {
    smd$COLclone[i] <- thisclone
    smd$finalClone[i] <- finalCloneDF$final[finalCloneDF$initial == smd$COLclone[i]]
  }
  rm(thisclone)

  if(smd$FILT[i] == TRUE) {
    smd$finalClone[i] <- NA
  }
}
#Samples without a clone ID were those that were filtered from the dataset
sum(is.na(smd$finalClone))
```

```
## [1] 530
```

```
sum(smd$FILT)
```

```
## [1] 530
```

```
length(which(is.na(smd$finalClone) & smd$FILT))
```

```
## [1] 530
```

```
nSex2 <- nSexGT2 <- nRegion2 <- c()
for(i in 1:max(smd$finalClone, na.rm = TRUE)) {
  thisclone <- which(smd$finalClone == i)
  if(length(thisclone) == 0) {
    nSex2[i] <- NA
    nSexGT2[i] <- NA
    nRegion2[i] <- NA
  } else {
    tmp <- smd[thisclone,]
    nSex2[i] <- length(unique(tmp$knownSEX[tmp$knownSEX != "" & !is.na(tmp$knownSEX)]))
    nSexGT2[i] <- length(unique(tmp$sexGT[tmp$sexGT != "undet" & !is.na(tmp$sexGT)]))
    nRegion2[i] <- length(unique(tmp$region))
    rm(tmp)
  }
}
```

Replicated samples

We don't want to have any replicates that are assigned to different final clones!

```
repIDs <- unique(sub("R","",smd$ugaID[which(smd$is.rep == TRUE)]))
check.rep.clone <- data.frame(ugaID = repIDs,
                              clone1 = NA,
                              clone2 = NA)

for(i in 1:length(repIDs)) {
  tmp.match <- c(repIDs[i], paste0(repIDs[i],"R"))
  check.rep.clone[i,-1] <- smd$finalClone[which(smd$ugaID %in% tmp.match)]
  rm(tmp.match)
}
check.rep.clone
```

##	ugaID	clone1	clone2
## 1	35-SGA	NA	NA
## 2	121-23	NA	NA
## 3	8-23	69	69
## 4	47-SGA	80	80
## 5	23-C	3	3
## 6	58-SGA	96	96
## 7	306-23	NA	NA
## 8	166-24	272	272
## 9	68-NGA	17	17
## 10	21-23	43	43
## 11	251-23	38	38
## 12	331-23	265	265
## 13	409-23	14	14
## 14	14-C	292	NA
## 15	241-23	1	1
## 16	358-24	33	33
## 17	280-23	NA	NA
## 18	50-23	4	4
## 19	358-23	37	37
## 20	5-24	262	262
## 21	134-24	NA	262
## 22	219-24	29	29
## 23	277-24	309	309
## 24	397-24	24	24
## 25	423-24	282	282
## 26	434-24	NA	NA
## 27	1-C	20	20
## 28	17-C	132	132
## 29	227-23	NA	NA
## 30	389-24	292	292
## 31	306-24	272	272
## 32	930-23	255	255
## 33	564-24	24	24
## 34	781-23	14	14
## 35	798-23	292	292
## 36	651-23	14	14
## 37	517-24	NA	NA
## 38	716-23	NA	NA

```
## 39 726-24      NA      NA
## 40 980-24     304     304
## 41 935-23      33      33
## 42 1306-24    316     316
## 43 567-24      24      24
## 44 881-24     288     288
## 45 2972-24    292     292
## 46 1384-24     NA      NA
## 47 933-24     135     135
## 48 1522-24    277     277
## 49 3058-24    316     316
## 50 2665-24    358     358
## 51 2869-24    272     272
## 52 2926-24     NA      NA
## 53 151-24      33      33
## 54 396-24     272     NA
## 55 424-23      NA      NA
## 56 191-24      NA      NA
## 57 466-23      NA      NA
## 58 286-23      NA      NA
## 59 500-24     316     316
## 60 311-23      NA      NA
```

```
noNAs <- check.rep.clone[which(rowSums(is.na(check.rep.clone))==0),]
nrow(noNAs)
```

```
## [1] 41
```

```
sum(noNAs$clone1 == noNAs$clone2)
```

```
## [1] 41
```

Good! We don't assign any replicated samples to separate clones!

Known Sex

As above, we should not have multiple known sexes per clone.

```
nSex2
```

```
## [1] 0 NA 1 1 1 0 0 0 NA NA 1 NA NA 0 1 NA 1 1 1 1 1 0 NA 0 NA
## [26] NA NA NA 0 NA NA NA 0 0 0 NA 0 0 NA 0 0 0 0 1 1 NA 1 1 0 1
## [51] 0 1 1 1 NA 1 0 1 1 1 NA 1 NA 0 NA 1 1 0 1 1 1 1 0 NA 1
## [76] 0 1 NA 1 1 1 1 1 0 0 1 1 1 1 1 1 1 NA 1 1 1 NA 1 1 NA
## [101] 1 1 0 NA 0 1 1 1 1 1 1 1 1 1 1 1 1 NA 1 1 1 1 1 1 1
## [126] 1 1 1 1 1 0 1 1 1 1 0 0 1 1 1 1 1 1 1 1 1 0 0 1 1
## [151] 1 0 0 1 1 1 0 1 0 1 1 0 0 1 1 1 1 0 0 1 0 1 1 0 NA
## [176] 0 0 NA 1 0 1 NA 1 0 0 1 1 1 NA NA 0 1 0 0 0 0 0 1 0 NA
## [201] NA 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 1 0 0 0
## [226] 0 0 0 NA 0 0 0 0 1 0 0 0 1 1 NA 0 0 0 0 NA 0 0 0 0 0
## [251] 0 0 0 NA 0 NA 0 NA NA 0 1 1 0 NA 0 0 NA NA 1 NA 0 1 NA NA 0
```

```
## [276] NA 0 NA NA 0 0 1 1 1 NA 0 0 0 NA NA 0 1 0 0 NA NA 1 1 1 0
## [301] 0 0 NA 0 NA 0 0 0 0 0 0 0 0 NA NA 1 0 0 0 0 0 0 0 0
## [326] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [351] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

```
sum(!is.na(nSex2))
```

```
## [1] 310
```

```
max(nSex2, na.rm = TRUE)
```

```
## [1] 1
```

Genotype-inferred Sex

And again, we would hope that there were relatively few cases of multiple genotype-inferred sexes within clones.

```
nSexGT2
```

```
## [1] 1 NA 1 1 1 1 1 2 NA NA 1 NA NA 2 1 NA 1 1 1 1 1 NA 1 NA
## [26] NA NA NA 1 NA NA NA 2 1 1 NA 1 1 NA 1 1 1 1 2 NA 1 1 1 1
## [51] 1 1 1 1 NA 1 0 1 1 1 NA 1 NA 1 NA 1 1 1 1 1 1 1 NA 1
## [76] 1 1 NA 1 1 1 1 1 1 1 1 1 1 1 1 1 NA 1 1 1 NA 1 1 NA
## [101] 1 1 1 NA 1 1 1 1 1 1 1 1 1 1 1 1 NA 1 1 1 1 1 1 1
## [126] 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 1
## [151] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 NA
## [176] 1 1 NA 1 1 1 NA 1 1 1 1 1 1 NA NA 1 1 1 1 1 1 1 1 NA
## [201] NA 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## [226] 1 1 1 NA 1 1 1 1 1 1 1 0 1 1 0 NA 1 1 1 1 NA 1 0 1 1
## [251] 1 1 1 NA 1 NA 1 NA NA 1 1 1 1 NA 1 1 NA NA 1 NA 1 1 NA NA 1
## [276] NA 1 NA NA 1 1 1 1 1 NA 1 1 1 NA NA 1 1 1 1 NA NA 1 2 1 1
## [301] 1 1 NA 1 NA 1 1 1 1 1 1 1 1 NA NA 1 1 1 1 1 1 1 1 1
## [326] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## [351] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

```
sum(!is.na(nSexGT2))
```

```
## [1] 310
```

```
max(nSexGT2, na.rm = TRUE)
```

```
## [1] 2
```

```
which(nSexGT2 > 1)
```

```
## [1] 8 14 33 45 298
```

```
#They do in a few cases...
#8, 14, 33, 45, 298
```

```
#Clone 8
cloneInQ <- 8
table(smd$sexGT[smd$finalClone == cloneInQ])
```

```
##
## F M
## 1 17
```

```
table(smd$knownSEX[smd$finalClone == cloneInQ])
```

```
##
##
## 9
```

```
#Kinship values for samples assigned to different clones by COLONY
quantile(compare.out$kinship[
  compare.out$colclone1 %in% smd$COLclone[which(smd$finalClone == cloneInQ)] &
  compare.out$colclone2 %in% smd$COLclone[which(smd$finalClone == cloneInQ)] &
  compare.out$iscloneCOL==FALSE])
```

```
##      0%      25%      50%      75%     100%
## 0.416667 0.458333 0.468750 0.483871 0.500000
```

```
#Clone 14
cloneInQ <- 14
table(smd$sexGT[smd$finalClone == cloneInQ])
```

```
##
## F M undet
## 58 2 2
```

```
table(smd$knownSEX[smd$finalClone == cloneInQ])
```

```
##
##
## 31
```

```
#Kinship values for samples assigned to different clones by COLONY
quantile(compare.out$kinship[
  compare.out$colclone1 %in% smd$COLclone[which(smd$finalClone == cloneInQ)] &
  compare.out$colclone2 %in% smd$COLclone[which(smd$finalClone == cloneInQ)] &
  compare.out$iscloneCOL==FALSE])
```

```
##      0%      25%      50%      75%     100%
## 0.354167 0.472222 0.484375 0.491935 0.500000
```

```

#Clone 33
cloneInQ <- 33
table(smd$sexGT[smd$finalClone == cloneInQ])

##
##      F      M undet
##      1      62      5

table(smd$knownSEX[smd$finalClone == cloneInQ])

##
##
## 24

#Kinship values for samples assigned to different clones by COLONY
quantile(compare.out$kinship[
  compare.out$colclone1 %in% smd$COLclone[which(smd$finalClone == cloneInQ)] &
  compare.out$colclone2 %in% smd$COLclone[which(smd$finalClone == cloneInQ)] &
  compare.out$iscloneCOL==FALSE])

##      0%      25%      50%      75%      100%
## 0.350000 0.481481 0.490741 0.500000 0.500000

#Clone 45
cloneInQ <- 45
table(smd$sexGT[smd$finalClone == cloneInQ])

##
## F M
## 5 1

table(smd$knownSEX[smd$finalClone == cloneInQ])

##
##      F
## 2 1

#Kinship values for samples assigned to different clones by COLONY
quantile(compare.out$kinship[
  compare.out$colclone1 %in% smd$COLclone[which(smd$finalClone == cloneInQ)] &
  compare.out$colclone2 %in% smd$COLclone[which(smd$finalClone == cloneInQ)] &
  compare.out$iscloneCOL==FALSE])

##      0%      25%      50%      75%      100%
##      NA      NA      NA      NA      NA

#Clone 298
cloneInQ <- 298
table(smd$sexGT[smd$finalClone == cloneInQ])

```

```
##
## F M
## 23 1
```

```
table(smd$knownSEX[smd$finalClone == cloneInQ])
```

```
##
## F
## 5 1
```

```
#Kinship values for samples assigned to different clones by COLONY
quantile(compare.out$kinship[
  compare.out$colclone1 %in% smd$COLclone[which(smd$finalClone == cloneInQ)] &
  compare.out$colclone2 %in% smd$COLclone[which(smd$finalClone == cloneInQ)] &
  compare.out$iscloneCOL==FALSE])
```

```
##      0%      25%      50%      75%     100%
## 0.384615 0.475000 0.490385 0.491667 0.500000
```

There's little ambiguity in any of these cases.

Region

Hopefully no cross-region recaptures!

```
nRegion2
```

```
## [1] 1 NA 2 1 1 1 1 1 NA NA 1 NA NA 1 1 NA 1 1 1 1 1 NA 1 NA
## [26] NA NA NA 1 NA NA NA 1 1 1 NA 1 1 NA 1 1 1 2 1 1 NA 1 1 1 1
## [51] 1 1 1 1 NA 1 1 1 1 1 1 NA 1 NA 1 1 1 1 1 1 1 1 NA 1 1 NA 1
## [76] 1 1 NA 1 1 1 1 1 1 1 1 1 1 1 1 1 1 NA 1 1 1 NA 1 1 NA
## [101] 1 1 1 NA 1 1 1 1 1 1 1 1 1 1 1 1 1 NA 1 1 1 1 1 1 1
## [126] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## [151] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 NA
## [176] 1 1 NA 1 1 1 NA 1 1 1 1 1 1 1 NA NA 1 1 1 1 1 1 1 1 NA
## [201] NA 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## [226] 1 1 1 NA 1 1 1 1 1 1 1 1 1 1 1 NA 1 1 1 1 NA 1 1 1 1
## [251] 1 1 1 NA 1 NA 2 NA NA 1 1 1 1 NA 1 1 NA NA 1 NA 1 1 NA NA 1
## [276] NA 1 NA NA 1 1 1 1 1 NA 1 1 1 1 NA NA 1 1 1 1 NA NA 1 1 1
## [301] 1 1 NA 1 NA 1 1 1 1 1 1 1 1 1 NA NA 1 1 1 1 1 1 1 1 1
## [326] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
## [351] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

```
which(nRegion2 > 1)
```

```
## [1] 3 43 257
```

```
smd$region[which(smd$finalClone == 3)]
```

```
## [1] "CGA" "CGA" "CGA" "CGA" "CGA" "CGA" "CGA" "CGA" "CGA"
## [10] "CGA-H" "CGA" "CGA" "CGA" "CGA" "CGA" "CGA" "CGA" "CGA"
## [19] "CGA" "CGA" "CGA" "CGA" "CGA" "CGA"
```

```
#Not really 2 regions, just historic sample
smd$region[which(smd$finalClone == 43)]
```

```
## [1] "CGA" "CGA" "CGA" "CGA" "CGA" "CGA" "CGA" "CGA" "CGA"
## [10] "CGA" "CGA" "CGA" "CGA" "CGA" "CGA" "CGA" "CGA" "CGA-H"
```

```
#Not really 2 regions, just historic sample
smd$region[which(smd$finalClone == 257)]
```

```
## [1] "CGA-H" "CGA" "CGA" "CGA" "CGA" "CGA" "CGA" "CGA" "CGA"
## [10] "CGA"
```

```
#Not really 2 regions, just historic sample
```

Known vs. Genotype-inferred Sex

Finally (again), comparing the known sex of unique genotypes (from direct captures) with that inferred for all samples within a clone...

```
#check for discrepancies between known and genotyped sex for all clones
discrepancy <- multknown <- c()
#for(i in 1:max(sampleMetadata$cloneID, na.rm = TRUE)) {
for(i in 1:max(smd$finalClone, na.rm=TRUE)) {
  thisclone <- which(smd$finalClone == i)
  if(length(thisclone) > 0) {
    tmp <- smd[thisclone,]
    tmp.knownSEX <- unique(tmp$knownSEX[tmp$knownSEX != "" & !is.na(tmp$knownSEX)])
    if(length(tmp.knownSEX) > 0) {
      if(length(tmp.knownSEX) > 1) {
        multknown[i] <- TRUE
        discrepancy[i] <- NA
      } else {
        multknown[i] <- FALSE
        if(sum(tmp$sexGT[tmp$sexGT != "undet"] != tmp.knownSEX) == 0) {
          discrepancy[i] <- FALSE
        } else {
          discrepancy[i] <- TRUE
        }
      }
    }
  }
  rm(tmp, tmp.knownSEX)
}
rm(thisclone)
}

sum(discrepancy, na.rm = TRUE) #Still only 3 cases with discrepancies...
```

```
## [1] 3
```

```
sum(multknown, na.rm = TRUE) #Still zero cases with multiple "known" sexes
```

```
## [1] 0
```

```
which(discrepancy == TRUE)
```

```
## [1] 45 144 298
```

```
#CloneIndex 45 - spotted above, direct capture agrees with majority  
table(smd$knownSEX[which(smd$finalClone == 45)])
```

```
##  
## F  
## 2 1
```

```
#CloneIndex 144 - spotted previously, direct captured cub  
table(smd$knownSEX[which(smd$finalClone == 144)])
```

```
##  
## M  
## 1
```

```
#CloneIndex 298 - spotted above, direct capture agrees with majority  
table(smd$knownSEX[which(smd$finalClone == 298)])
```

```
##  
## F  
## 5 1
```

Assign Final Sex

Using the combined information across samples within a clone, assign sex to each unique genotype.

```
smd$finalSEX <- rep(NA, nrow(smd))  
smd$knownSEX[is.na(smd$knownSEX)] <- ""  
for(i in 1:nrow(smd)) {  
  tmp <- smd[which(smd$finalClone == smd$finalClone[i]),]  
  if(length(tmp$knownSEX[tmp$knownSEX != ""]) > 0) {  
    smd$finalSEX[i] <- unique(tmp$knownSEX[tmp$knownSEX != ""])  
  } else if(length(tmp$sexGT[tmp$sexGT != "undet"] > 0)) {  
    smd$finalSEX[i] <- names(which.max(table(  
      tmp$sexGT[tmp$sexGT != "undet"])))  
  }  
  rm(tmp)  
}  
  
table(smd$finalSEX)
```

```
##  
## F M  
## 868 803
```

```
#Is the final called sex ever different from known sex? Shouldn't be
smd[which(smd$finalSEX != smd$knownSEX & smd$knownSEX != ""),]
```

```
## [1] extID      ugaID      is.SCR     is.rep     week       year
## [7] snare       utmSnare   fqfile     knownSEX   sampSOURCE region
## [13] nSNPgt      vcfID      FILTER     GTservice  sexGT      COLclone
## [19] finalClone finalSEX
## <0 rows> (or 0-length row.names)
```

```
#Is the final called sex ever different from genotyped sex - should be a few
# 63-DC: den check cub, called a female by GT, M in the field
# 497-24: capture in the matches, a F
# 212-23 & 271-23: no captures, but majority F
# 774-24: capture in the matches, F
# 5-23: majority M no captures in matches
# 655-23: majority M no captures in matches
smd[which(smd$finalSEX != smd$sexGT & smd$sexGT != "undet"),]
```

```
##
##              extID  ugaID  is.SCR  is.rep  week  year  snare
## 71              UGA_173101_P015_WF11  63-DC  FALSE  FALSE  <NA>  2024  <NA>
## 209             UGA_173101_P017_WB05  497-24   TRUE  FALSE    2  2024  HS 048
## 549             UGA_173101_P020_WH03  212-23   TRUE  FALSE    6  2023  HS 027
## 550             UGA_173101_P020_WH04  271-23   TRUE  FALSE    6  2023  HS 025
## 629             UGA_173101_P021_WF11  774-24   TRUE  FALSE    2  2024  HS 084
## 828  GTseek_024_384i5_001_P098.001_5-23   5-23   TRUE  FALSE    6  2023  HS 012
## 962  GTseek_024_384i5_135_P098.001_655-23 655-23   TRUE  FALSE    6  2023  HS 012
##
##          utmSnare
## 71          <NA>
## 209 261267x3598930
## 549 264464x3592495
## 550 265276x3591787
## 629 267164x3597536
## 828 258412x3586648
## 962 258412x3586648
##
##                                     fqfile
## 71  RAPiD-Genomics_F534_UGA_173101_P015_WF11_i5-502_i7-51_S14809_L003_R1_001.fastq
## 209 RAPiD-Genomics_F534_UGA_173101_P017_WB05_i5-512_i7-113_S14946_L003_R1_001.fastq
## 549  RAPiD-Genomics_F566_UGA_173101_P020_WH03_i5-505_i7-88_S3977_L001_R1_001.fastq
## 550  RAPiD-Genomics_F566_UGA_173101_P020_WH04_i5-505_i7-73_S3978_L001_R1_001.fastq
## 629  RAPiD-Genomics_F566_UGA_173101_P021_WF11_i5-501_i7-51_S71_L003_R1_001.fastq
## 828                                     <NA>
## 962                                     <NA>
##
##      knownSEX  sampSOURCE  region  nSNPgt      vcfID  FILTER  GTservice
## 71          M    Den Check    CGA    1198  UGA_173101_P015_WF11  FALSE      LGC
## 209          Hair Snare    CGA    1224  UGA_173101_P017_WB05  FALSE      LGC
## 549          Hair Snare    CGA    1218  UGA_173101_P020_WH03  FALSE      LGC
## 550          Hair Snare    CGA    1215  UGA_173101_P020_WH04  FALSE      LGC
## 629          Hair Snare    CGA    1209  UGA_173101_P021_WF11  FALSE      LGC
## 828          Hair Sample    CGA     104              GTS_5-23  FALSE    GTSeek
## 962          Hair Sample    CGA     116              GTS_655-23  FALSE    GTSeek
##
##      sexGT  COLclone  finalClone  finalSEX
## 71      F      144      144      M
## 209     M       30      298      F
```

## 549	M	14	14	F
## 550	M	14	14	F
## 629	M	45	45	F
## 828	F	254	8	M
## 962	F	270	33	M

Compare kinship coefficients

As one final check of the reliability of our assignments, we'll look at the distribution of KING kinship coefficients for samples from the same vs. different individual. We'll do this in two phases, first for technical replicates (accounting for inferred recaptures among replicated hair samples) and then for all SCR samples.

Technical replicates

First for only the technically replicated samples. Note that it's possible that we have recaptures represented in these, so we'll want to identify unique individuals from this set (as inferred above) before plotting the distribution within / between technical replicates.

```
#Create an object for within vs. between clone comparisons - technical replicates
king.allREP <- kingrel[
  which(kingrel$id1 %in% smd$vcfID[which(smd$is.SCR == TRUE &
                                         smd$is.rep == TRUE)] &
        kingrel$id2 %in% smd$vcfID[which(smd$is.SCR == TRUE &
                                         smd$is.rep == TRUE)]),]

king.allREP$clone1 <- king.allREP$clone2 <- NA
for(i in 1:nrow(king.allREP)) {
  king.allREP$clone1[i] <- smd$finalClone[which(smd$vcfID == king.allREP$id1[i])]
  king.allREP$clone2[i] <- smd$finalClone[which(smd$vcfID == king.allREP$id2[i])]
}

#Same Clone
range(king.allREP$kinship[which(king.allREP$clone1 == king.allREP$clone2)])
```

```
## [1] 0.395833 0.500000
```

```
#Different Clone
range(king.allREP$kinship[which(king.allREP$clone1 != king.allREP$clone2)])
```

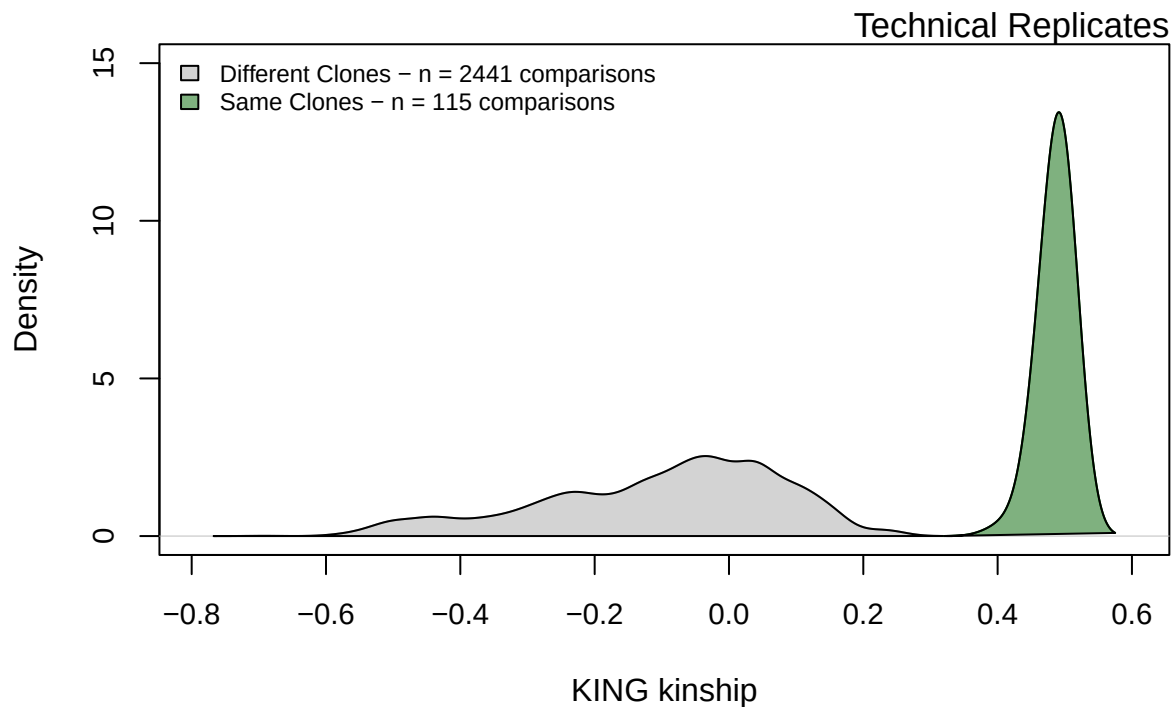
```
## [1] -0.692308 0.257812
```

```
#Plot the distributions
library(scales)
same.clone <- density(king.allREP$kinship[which(king.allREP$clone1 == king.allREP$clone2)], bw = 0.025)
diff.clone <- density(king.allREP$kinship[which(king.allREP$clone1 != king.allREP$clone2)], bw = 0.025)
plot(diff.clone, xlim = c(min(king.allREP$kinship)-0.1, max(king.allREP$kinship)+0.1),
     ylim = c(0, 15), lwd = 1, col = "darkgrey", cex.axis = 0.9,
     xlab = "KING kinship", ylab = "Density", main = "")
polygon(diff.clone, col = alpha("darkgreen", 0.5))
lines(same.clone, col = "darkgreen")
polygon(same.clone, col = alpha("darkgreen", 0.5))
```

```

legend("topleft", bty = "n", cex = 0.75,
      fill = c(alpha("darkgrey", 0.5), alpha("darkgreen", 0.5)),
      legend = c(paste("Different Clones - n =",
                       length(which(king.allREP$clone1 != king.allREP$clone2)),
                       "comparisons"),
                 paste("Same Clones - n =",
                       length(which(king.allREP$clone1 == king.allREP$clone2)),
                       "comparisons")))
mtext("Technical Replicates", cex = 1.1, adj = 1)

```



All SCR samples

Now for all SCR samples. Note the *very* large number of pairwise comparisons represented here.

```

## All SCR
#Create an object for all SCR samples (that were assigned to a finalClone)
keepVCfList <- smd$vcfID[which(smd$is.SCR == TRUE &
                             !is.na(smd$finalClone) &
                             !is.na(smd$finalSEX))]
kingSCR <- kingrel[which(kingrel$id1 %in% keepVCfList & kingrel$id2 %in% keepVCfList),]
kingSCR$clone1 <- kingSCR$clone2 <- NA
for(i in 1:nrow(kingSCR)) {
  kingSCR$clone1[i] <- smd$finalClone[which(smd$vcfID == kingSCR$id1[i])]
  kingSCR$clone2[i] <- smd$finalClone[which(smd$vcfID == kingSCR$id2[i])]
}

```

```

}

#Same Clone
range(kingSCR$kinship[which(kingSCR$clone1 == kingSCR$clone2)])

## [1] 0.233871 0.500000

sum(kingSCR$kinship[which(kingSCR$clone1 == kingSCR$clone2)] < 0.45)

## [1] 2037

#Different Clone
range(kingSCR$kinship[which(kingSCR$clone1 != kingSCR$clone2)])

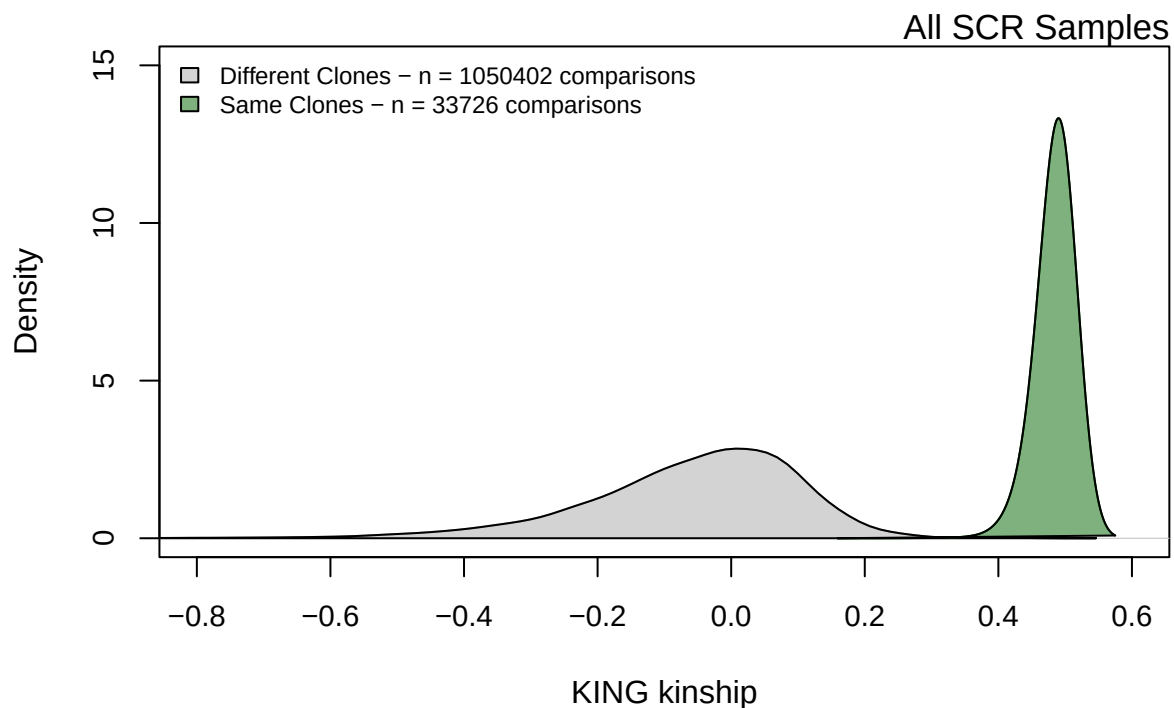
## [1]      -Inf 0.471154

kingSCR$nsnp[which(kingSCR$kinship >= 0.45 & kingSCR$clone1 != kingSCR$clone2)]

## [1] 958 965 958 783 964 945 940 964 73 91 59 85 85 74 89 88 93 86 89
## [20] 86 88 89 73 74 64 68 89 73 68 86 89 87 88 74 86 88 74 67
## [39] 73 59 85 89 89 88 89 89 88 88 88 90 68 73 73 88 89 88 74
## [58] 89 67 87 91 88 66 88 87 73 88 73 89 73 87 89 74 89 87 89
## [77] 74 74 74 73 74 87 89 66 89 87 67 66 68 68 57 68 84 91 64

same.cloneALL <- density(kingSCR$kinship[which(kingSCR$clone1 == kingSCR$clone2)], bw = 0.025)
diff.cloneALL <- density(kingSCR$kinship[which(kingSCR$clone1 != kingSCR$clone2)], bw = 0.025)
plot(diff.cloneALL, xlim = c(-0.8, 0.6),
     ylim = c(0, 15), lwd = 1, col = "darkgrey", cex.axis=0.9,
     xlab = "KING kinship", ylab = "Density", main = "")
polygon(diff.cloneALL, col = alpha("darkgrey", 0.5))
lines(same.cloneALL, col = "darkgreen")
polygon(same.cloneALL, col = alpha("darkgreen", 0.5))
legend("topleft", bty = "n", cex = 0.75,
     fill = c(alpha("darkgrey", 0.5), alpha("darkgreen", 0.5)),
     legend = c(paste("Different Clones - n =",
                     length(which(kingSCR$clone1 != kingSCR$clone2)),
                     "comparisons"),
               paste("Same Clones - n =",
                     length(which(kingSCR$clone1 == kingSCR$clone2)),
                     "comparisons")))
mtext("All SCR Samples", cex = 1.1, adj = 1)

```



Final Summary

How many total genotypes do we see in the dataset? How many from each region? How many male and female genotypes in our SCR datasets?

```
#Loop through all clones and record clone ID, size of clone, final sex, etc.
sumtab <- data.frame(cloneID=unique(smd$finalClone[!is.na(smd$finalClone)]),
                     size=NA, finalSEX=NA, region=NA)
for(i in 1:nrow(sumtab)) {
  sumtab$size[i] <- length(which(smd$finalClone == sumtab$cloneID[i]))
  sumtab$finalSEX[i] <- smd$finalSEX[which(smd$finalClone == sumtab$cloneID[i])[1]]
  sumtab$region[i] <- smd$region[which(smd$finalClone == sumtab$cloneID[i])[1]]
}

#Total unique genotypes
nrow(sumtab)
```

```
## [1] 310
```

```
#By sex
table(sumtab$finalSEX)
```

```
##
## F M
## 153 153
```

```

#By region
table(sumtab$region)

##
##   CGA CGA-H   NGA   SGA
##   232   15   35   26

#CGA hair samples from this study only
hair.smd <- smd[which(smd$is.SCR == TRUE & smd$region == "CGA" & smd$year > 2020),]
#Total hair samples
nrow(hair.smd)

## [1] 1958

#Total hair samples successfully assigned to a clone / not filtered
nrow(hair.smd[which(hair.smd$FILT == FALSE),])

## [1] 1477

nrow(hair.smd[!is.na(hair.smd$finalClone),])

## [1] 1477

nrow(hair.smd[which(!is.na(hair.smd$finalClone) & hair.smd$FILT == FALSE),])

## [1] 1477

#Total unique genotypes from hair samples
length(unique(hair.smd$finalClone[!is.na(hair.smd$finalClone)]))

## [1] 169

#Total unique female genotypes from hair samples
length(unique(hair.smd$finalClone[which(hair.smd$finalSEX == "F" &
!is.na(hair.smd$finalClone))]))

## [1] 88

#Total male genotypes from hair samples
length(unique(hair.smd$finalClone[which(hair.smd$finalSEX == "M" &
!is.na(hair.smd$finalClone))]))

## [1] 77

```

Save the output!

Write the sampleMetaData file out to a csv for use in building the capture history!

```
write.csv(smd, "../sampleinfo/sampleMetaData_cloneID.csv", quote = FALSE, row.names = FALSE)
```